Polaris

Fixus inter mutabilia

The map of the system

A reference implementation of a post-quantum, issuer-unlinkable,
compulsion-resistant identity-token system,
read from the centre outward, at version 9.355

Egor Khaklin
Seton Hill University · September 2026

About this version

Version 1 of this report (`polaris_project_report.pdf`, preserved unchanged beside this file) described Polaris as a database-design project: twelve tables around one credential, an entity-relationship model, a lifecycle state machine, and the argument that identity data must be post-quantum from the first day. That description remains the historical record. Since then the repository has grown from twelve tables to thirty-six, from one application to a federation protocol, a transparency layer and an institutional exchange fabric, and from zero machine-checked invariants to 191. Version 2 describes the system as it stands at tag `v9.345` (9 September 2026), and it is written for a reader who has never opened the repository.

Two dates, stated plainly. The body was written from the tree at `v9.345` and every count in it was measured there. Between that tag and `v9.355` the repository shipped Phase P9, which closed five absences this paper had recorded as open: a key the holder controls, a prover that runs on the holder's device, a per-verifier nullifier, a per-relying-party handle, and a delegated agent grant. Leaving those sections as they were would have made the paper a description of a system that no longer exists, so Section 8 and Section 17, the appendix status ledger and the figures they carry were rewritten at `v9.355`. The counts elsewhere are still the `v9.345` measurements and are labelled as such wherever they appear; a reader checking one against the current tree should expect it to have grown. Nothing that ran at `v9.345` stopped running.

The paper keeps what Version 1 got right: the token-centred geometry of the schema, used here as the centre of every map, and the lifecycle machine, with one correction, since the trigger Version 1 recommended for enforcing it now exists. The post-quantum argument is kept, as Appendix E summarises. Everything else is written fresh from the tree.

Four marks

Every capability carries one of four marks, so that code that runs is never confused with an intention.

IMPLEMENTED A path that runs (at `v9.345`, or at `v9.355` in the two rewritten sections), exercised by a test or a drill in continuous integration, and pinned by an invariant check that fails the build if it stops being true.

EXPERIMENTAL A path that runs but is limited in a way the repository itself names: not covered by CI, notional in scale, or dependent on a backend that CI simulates.

EXTERNAL VALIDATION REQUIRED A property the repository implements and tests internally but that no external party has yet audited, attacked, deployed or independently re-implemented: no penetration test, cryptographic audit, pilot or third-party implementation has occurred.

PROPOSED A proposal: a roadmap row, a design note, or an idea that exists only in this paper. It is not code.

Numbers were measured on the tree at `v9.345` by the methods in Appendix C, not copied from earlier documents. Where the repository's own documents disagree with its code, the code wins and the disagreement is recorded in the author's notes that accompany this version.

The analogy, and its limit

The paper explains Polaris through one sustained analogy: a walled town. The credential record is the town hall's register; the constraints are the wall; the endpoints are gates; the transparency logs are watchtowers; independent witnesses stand outside the wall watching the towers; other authorities are other towns, joined by roads that run one way; and above the whole town stand laws that bind even the town itself. The analogy describes arrangement and sight lines, not political

centralisation: Polaris is federated per authority, with no central database, root or verification service, and every recurrence of the analogy returns to the actual mechanism by its real name.

One thesis

The system is an attempt to build identity infrastructure in which important claims do not have to be trusted merely because a central application says they are true. Authority, state, history, privacy boundaries and the system's own behaviour are progressively exposed to independent verification and constrained by layers that do not share each other's assumptions. Each layer exists because the layer inside it could be entirely correct and still fail to prove what a society needs to know. That progression is the spine of the paper.

Contents

1 The problem

A person in the United States carries six to eight credentials that do not talk to each other: a driver's licence, a passport, a Social Security card, a Real ID, a voter registration, an insurance card. Each is a separate artifact, signed by a separate authority to a separate standard, with no shared revocation path and no shared audit trail. Version 1 of this report set out to model their consolidation into one active credential record per person, verified through context-scoped events. That is the easy half of the problem, and it is still the problem's shape.

The hard half is what happens when the system works. An identity system that succeeds becomes the doorway to everything, and a doorway is a place where power concentrates. Three failure modes follow, and every layer of Polaris exists to answer one of them.

The system lies about the past. An operator, or an insider with database access, revokes a token and later erases the record; issues a credential under procedures the public was never told of; or shows one history to an auditor and another to a court. A national identity system cannot be allowed to retroactively rewrite what it did.

The system watches. Every verification is a fact about where a person was and what they were doing. A log of those facts is a surveillance backbone whether or not anyone intended one, and it stays one after the operator's intentions change.

The system coerces. A holder can be compelled: a border officer, an employer, or a thief can force a credential to be presented, and a doorway that every service requires can force a population to enrol. An identity system is also the most efficient instrument of exclusion ever built, if the list of who is not in it can be produced on demand.

The conventional answer to all three is policy: rules about retention, access, audit and consent, enforced by the people who run the system. Polaris's constitution (`MISSION.md`) takes the position that policy is worthless as the only thing holding a promise up, because the people who run the system are exactly the ones the promise is meant to bind. The promise has to be enforced where it cannot be talked around: in the schema, in triggers, in unique indexes, in check constraints, in signatures that anyone can verify without asking the system, and in logs that other people watch. The application is not the security boundary. The database is, and the layers around the database exist to make even the database's claims checkable by strangers.

1.1 Why post-quantum from the first day

Version 1 argued, through Mosca's inequality, that identity data has a secrecy horizon measured in generations, that migration of a national system takes a decade, and that a credential system deployed on classical cryptography in 2026 is therefore a deferred compromise rather than a working system. That argument is unchanged and is summarised in Appendix E. At `v9.345` the default signing algorithm is ML-DSA-65 (FIPS 204), ML-DSA-87 is accepted beside it, the zero-knowledge proof system is hash-based with no trusted setup, and the public TLS edge negotiates a hybrid post-quantum key exchange with modern clients. Which surfaces remain classical is stated plainly in Section 13 and in the repository's posture ledger.

1.2 The progression this paper follows

Each of the following sentences names a layer, and each is true of the layer before it.

- A credential's authenticity is not its current authorization. A signature stays genuine forever; the authority under it can be withdrawn this afternoon.
- Current authorization is not privacy. A verifier can learn that a credential is valid and still learn far more than it needed to.

- Privacy is not freedom from coercion. A system can know almost nothing about a person and still be the doorway every service demands.
- Federation is not institutional independence. Two instances of the same code run by the same author prove that the protocol works, not that independent institutions run it.
- A transparency log is not protection from a split view. One observer can be shown a private fork of history and never know.
- A green test is not proof that the test asked the right question.

The last sentence is the one the project has learned most expensively, and Section 15 is about it.

2 How to read this map

Polaris is not one enormous table or one enormous server. It is a very small credential core with deliberately bounded structures arranged around it, each of which constrains, protects, observes, verifies, records or limits something about the structure inside it. The paper reads outward from the core, ring by ring (Figure 1), and the reader should be able to zoom: from the person, to the credential, to the cryptography that authenticates it, to the schema rules that bound it, to the authority that issued it, to the parties that rely on it, to the federation of authorities, to the witnesses that watch them, to institutional exchange, to the operations that keep it alive, to the society it sits inside, and to the environment of software agents it is preparing for.

2.1 The questions asked of every layer

For each major component the paper answers the same questions, in a card of the form used throughout. What is this? Why does it exist? What problem does it solve? What does it trust? What does it refuse to trust? What can it see? What is it deliberately prevented from seeing? What happens if it fails? What part of Polaris checks it? Is it needed now, or is it preparation for a future environment? How does it connect to the layer inside it and the layer outside it? A reader who can answer those questions for every ring has the mental model this paper exists to build.

2.2 The town

IN THE TOWN The credential record is the register kept in the town hall: one entry per person, the claim that everything else is organised around. The wall is the schema: triggers, check constraints and unique indexes that decide which states may exist at all, and which bind every visitor, every official and every restore from backup alike. The gates are the endpoints through which a claim enters or leaves: issuance, verification, login, exchange. The watchtowers are the transparency logs, and the witnesses stand outside the wall, cosigning what the towers publish and shouting when two towers disagree. Other towns are other authorities, each with its own hall and its own key, joined by roads that run in one direction and for one purpose. Above everything stand laws that the town itself cannot repeal by a vote of its officials.

The map of the town, with the technical name beside every civic one, is Figure 5 in Section 4. The analogy will recur in a shaded box like the one above, and every recurrence is followed by the mechanism it stands for.

2.3 Trust and refusal

The most useful habit for reading Polaris is to ask, of every layer, what it refuses to take on faith from the layer below. The schema refuses to trust the application. The detached verifier refuses to trust the issuer's server. The second witness refuses to trust the first implementation. The

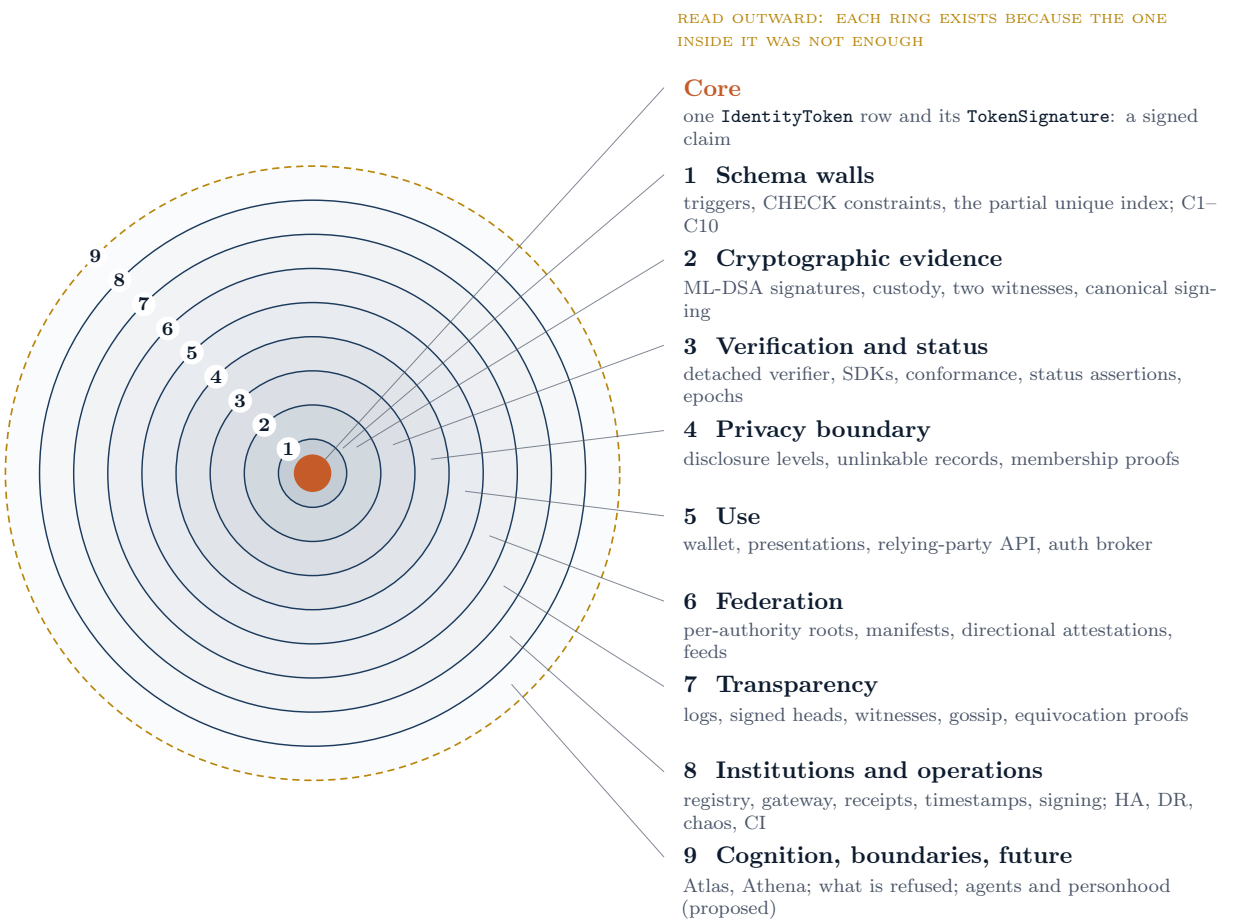


Figure 1: The concentric map of Polaris. The core is one credential record and its signature. Each ring names the actual components on it, and the reading order of the paper follows the rings outward. The outermost ring is dashed because part of it does not yet exist: cognition and boundaries are present, the agent environment is a proposal.

relying party refuses to trust a transitive chain of authorities. The witness refuses to trust the log's own head. The check layer refuses to trust a check that cannot detect its own violation. Appendix B tabulates the refusals as a ladder; Sections 3 through 13 explain each one.

3 The core: one credential, one signature, one person

3.1 What the credential is

At the centre of Polaris is one row of the table `IdentityToken` and at least one row of `TokenSignature`. The token row records a unique `token_value`, a physical serial, the person it belongs to, the agency that issued it, the algorithm it was issued under, its predecessor if it replaced an earlier token, and its status. The signature row records the issuing agency's ML-DSA signature over the SHA3-256 digest of the token value, together with the public key that produced it, so that verification is self-contained and survives the retirement of the signing key. A token without a signature row cannot exist: a database trigger refuses any write that would leave one without a non-deprecated signature. The credential is therefore not a document but a signed claim: *this authority issued this token to this person.* IMPLEMENTED

What the holder holds is the authenticity pack: the token value, the signature and the issuer's public key, exported by the application and kept as a file by the reference wallet

(`scripts/polaris-wallet.py`). Anyone can check the pack with a standard ML-DSA library and no Polaris code (Section 6). The physical token that Version 1 modelled, a hardware card with on-device biometric binding, remains modelled and not manufactured: the schema carries the serial, the binding type and the device bindings, and no card exists. PROPOSED for the hardware; IMPLEMENTED for the record.

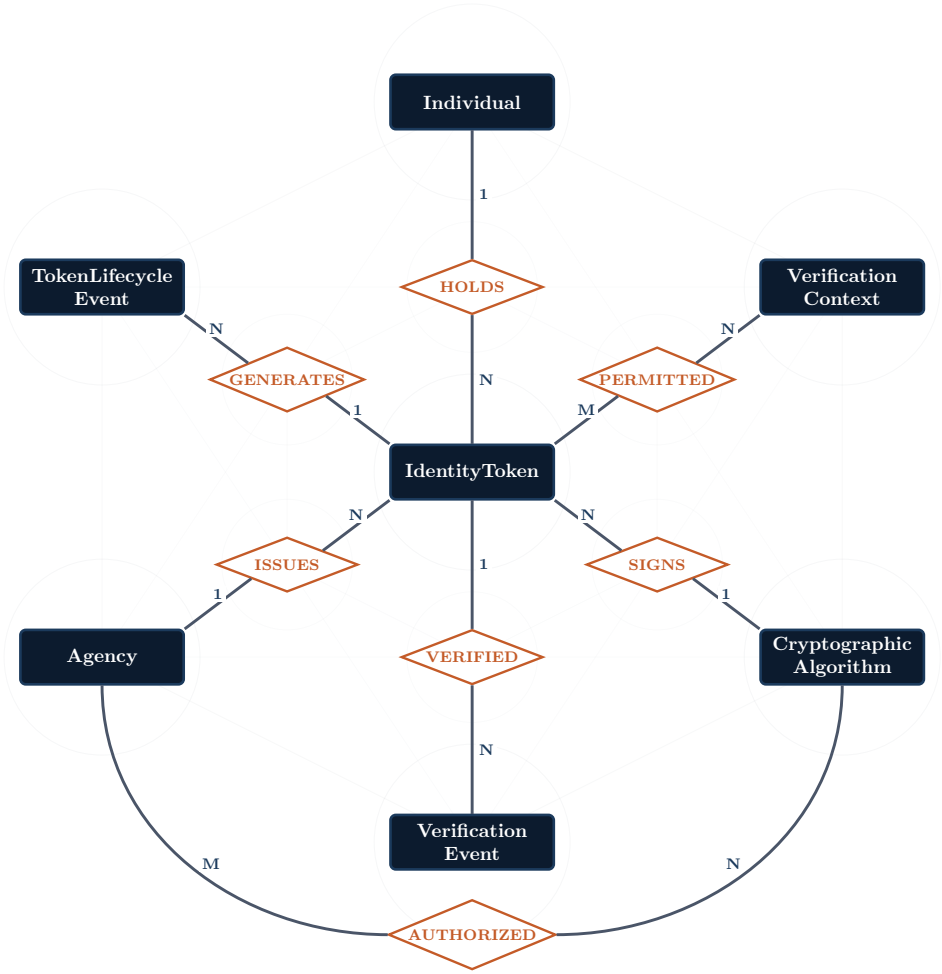


Figure 2: The core entity geometry, unchanged from Version 1. Seven entities around `IdentityToken`: the person who holds it, the agency that issues it, the algorithm that signs it, the contexts it may be verified in, and the two append-only event logs it generates. The many-to-many relationships resolve to the junction tables `AgencyAlgorithmAuth` and `TokenPermission`. This geometry is the constant of the paper: Section 4 draws the other twenty-nine tables as rings around it.

3.2 What the credential is not

The constitution's tenth constraint, C10, states that identity attestation never carries spending authority: there is no monetary claim in the schema, and a check fails the build if one appears. The reason is stated as a failure mode, not a preference. A credential that is also money inherits programmability: every constraint anyone wants to place on spending accretes onto the identity token, until an administrative paperwork error becomes an existential bank-balance error, and until the system can be told that a person may not buy fuel. The same refusal covers reputation, behaviour and profiles. Polaris answers one question, *is this token valid for this context right now*, and it declines to answer *should this person be allowed to do X*. That second question belongs to the relying party, outside the boundary. IMPLEMENTED

3.3 One active token per person

A person may hold several provisioned tokens but only one may be `ACTIVE`. Two active tokens for one person would let one token authorise a transaction and the other repudiate it. The rule is C3, and it is enforced by a partial unique index, `uq_one_active_per_person`, on the person's identifier where the status is active. Two operators activating two reserve tokens for the same holder at the same moment find that exactly one of them succeeds, and the test that proves it runs with real threads against a live database (C9). IMPLEMENTED

3.4 The lifecycle

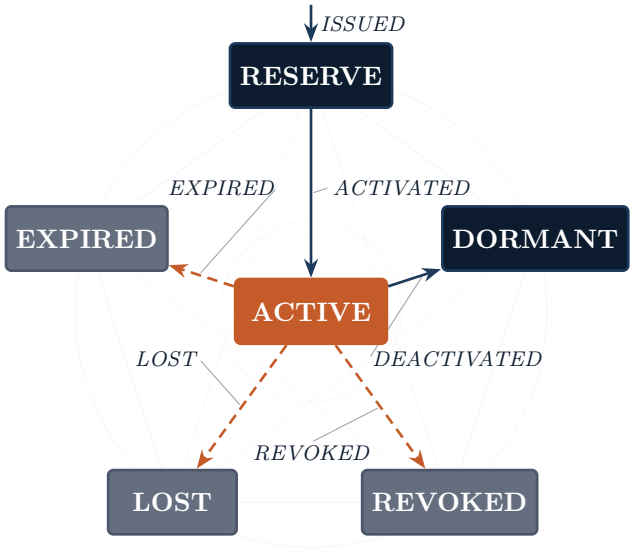


Figure 3: The token lifecycle machine, unchanged from Version 1. `ACTIVE` is the only operational state. A token enters at `RESERVE`, is activated, and leaves through one of three terminal states or into `DORMANT` when a successor takes over. At `v9.345` the legal transitions are enforced by the trigger `trg_token_state_machine`, and every transition writes a lifecycle event automatically.

Version 1 recommended enforcing the transitions in a trigger and left it as future work. At HEAD the trigger exists, and a second trigger writes a `TokenLifecycleEvent` row for every status change, including a change made by hand from a database shell. Succession is by reference, never by overwrite: a replacement token points at its predecessor and the old token stays in the database in its terminal state. Lost tokens stay lost, and their data is not erased. IMPLEMENTED

3.5 Entering, leaving and returning

Three mechanisms surround the core so that the schema cannot be turned against the holder at the edges of the lifecycle.

Entering: tiered enrolment. Every person has an enrolment status drawn from five values, and `EXEMPT` is a first-class one: a recognised way to take part in civic life without a token, whether for biometric incompatibility, religious exemption or an alternative path. The asymmetry is deliberate. Recording an exemption is one row; enumerating the not-enrolled is possible but unhelped by any function, and it is audited when done. Nothing is derived from token state, so an administrative revocation never silently becomes a person leaving civic life. IMPLEMENTED

Leaving: issuer discretion. The worst failure of an identity system is not a forged token but an authority revoking a population's tokens at scale. A trigger bounds how fast one agency can revoke its own tokens (five percent of its outstanding base in thirty days by default) and

above that a co-signer from a different agency is required, recorded in the audit trail. The bound converts a surprise into a trend that reporting can see. It does not stop slow abuse under the ceiling, and says so. IMPLEMENTED

Returning: the recovery ceremony. A fire takes the wallet and the reserve together. Recovery is mandatory and is also an impersonation path, so it is made expensive without being made impossible: a two-phase ceremony with a 48-hour cool-down, three independent out-of-band channels, an approver who cannot be the requester, and admin-only completion, each encoded as a check constraint on `RecoveryRequest`. The holder is without identity for the cool-down; the substitute credential a production system would want for that window is not built. IMPLEMENTED for the ceremony, PROPOSED for the interim credential.

3.6 Compulsion: the duress code

The vocation above the ten constraints is anti-coercion: no person may be compelled to renounce, transfer or surrender their identity against their will. The mechanism that gives that sentence something to stand on is old and comes from banking: a second secret that looks exactly like success to whoever is watching, while an alert goes out through a channel the watcher cannot see. A holder may enrol a duress code. When it is presented, the verification proceeds identically on every operator-visible surface, the same page, the same flash message, the same event row, and a `DuressEvent` row is appended to a table the operator's screens never join, from which a pager fires at the highest severity. The comparison is constant-time and the work is paid on both paths. IMPLEMENTED as a tested property of the modelled flow; EXTERNAL VALIDATION REQUIRED as a side-channel guarantee, because no external party has measured it.

Layer card: the credential core

What it is	One `IdentityToken` row and its `TokenSignature` rows: a signed claim that an authority issued this token to this person, in a lifecycle with one active state.
Why it exists	Every other structure in Polaris organises itself around this claim. It is kept as small as possible so that the rest can be bounded.
What it trusts	The issuing agency's signing key, held behind a custody interface (Section 5); the schema's constraints.
What it refuses	Application code as a source of truth: the state machine, uniqueness and audit are decided by the database. Money, reputation and profiles: C10.
What it can see	The holder's legal name, date of birth and country; the issuing agency; the algorithm; the lifecycle.
What it cannot see	A biometric template (never stored), a verification graph (Section 8), a spending history (never modelled).
If it fails	A forged core is a forged signature, which the two-witness verification and the detached verifier refuse; a duplicated core is refused by the unique index; a rewritten core is refused by the append-only triggers.
Checked by	`check_c1c10_objects_resolve`, the C3 property tests, the concurrency suite, the attack suite's forgery and tamper cases.
Now or later	Now. It is the part of Polaris that has existed since Version 1.
Inside and outside	Inside: the person, who exists outside the software. Outside: the wall of constraints (Section 4).

IN THE TOWN The town hall keeps one register. Each entry says who a person is, which official entered it, and under which seal. The register cannot be edited, only appended to; a second entry for the same person is refused at the desk; and the seal is one that every reader can check against the official's published mark without asking the hall.

4 The walls: the schema as the security boundary

4.1 Why the database is not plumbing

In most systems the database stores what the application decided. In Polaris the database decides, and the application is one client among several: the operator's web application, the command-line tool, a bulk loader, a database shell, a restore from backup. A rule that lives in application code binds only the clients that run that code. A rule enforced by a trigger, a check constraint or a unique index binds every client, survives every restore, and cannot be bypassed by the next caller, because the application role has no data-definition privilege with which to remove it. The governing sentence of the architecture is *by construction*: the system cannot be operated in a way that violates the constraint, because the constraint is enforced before any application code runs. IMPLEMENTED

The design principle behind it is older than Polaris: make invalid states impossible to represent rather than asking developers not to create them. A verification event that records a token identifier at the zero-knowledge disclosure level is not an application bug to be caught in review. It is a privacy violation that must never be storable, so it is a check constraint on the row.

4.2 The ten constraints and where each lives

The constitution names ten hard constraints in two tiers. Five are constitutional, rights guarantees that may not be relaxed without an amendment recorded in the changelog; five are engineering invariants, the disciplines that keep the first five true under load and attack. The hierarchy is one line deep, vocation, then constitutional constraints, then engineering invariants, and the constitution states that no deeper apparatus governs them.

Table 2: The ten constraints at `v9.345`, their tier, and the object that enforces each. The table is kept honest against the code by `check_c1c10_objects_resolve`, which fails the build if a named object no longer exists.

#	Constraint	Tier	Enforced by
C1	The audit trail is append-only; history is never rewritten or deleted	Constitutional	Triggers reject UPDATE and DELETE on every audit table (`reject_audit_modification`)
C2	A zero-knowledge verification stores no token identifier	Constitutional	Bidirectional CHECK `chk_disclosure_token_consistency`
C3	One person holds at most one ACTIVE token	Constitutional	Partial unique index `uq_one_active_per_person`
C4	Failed-login counting is atomic	Engineering	Single-statement `UPDATE ...RETURNING`
C5	No inline scripts; the content security policy is `script-src 'self'`	Engineering	Response header, verified per route
C6	Disclosure level is enforced server-side; a client cannot upgrade what it learns	Constitutional	Server code paired with redaction tests; the C2 CHECK behind it
C7	No hardcoded cryptography; algorithms are rows in a registry	Engineering	Foreign key to `CryptographicAlgorithm`
C8	Every map and API aggregate is bounded	Engineering	Hard caps in the SQL functions and routes
C9	Concurrency claims are tested with real threads	Engineering	Threaded suites against a live database
C10	Identity is not money; the schema carries no monetary claim	Constitutional	Structural absence, pinned by a check

The repository also records a structural claim about the ten: that the set is closed, so that an eleventh constraint requires either replacing one or extending the framework, and that the

constraints depend on one another, so that removing any one ripples through the rest. The dependency walk (remove C10 and every other constraint protects a financial-surveillance system instead; remove C1 and every other constraint's enforcement becomes retroactively deniable) is the argument for treating them as a graph rather than a list. Athena, in Section 14, holds the map from each constraint to the exact live mechanism that enforces it, and a check fails the build if a mechanism drifts.

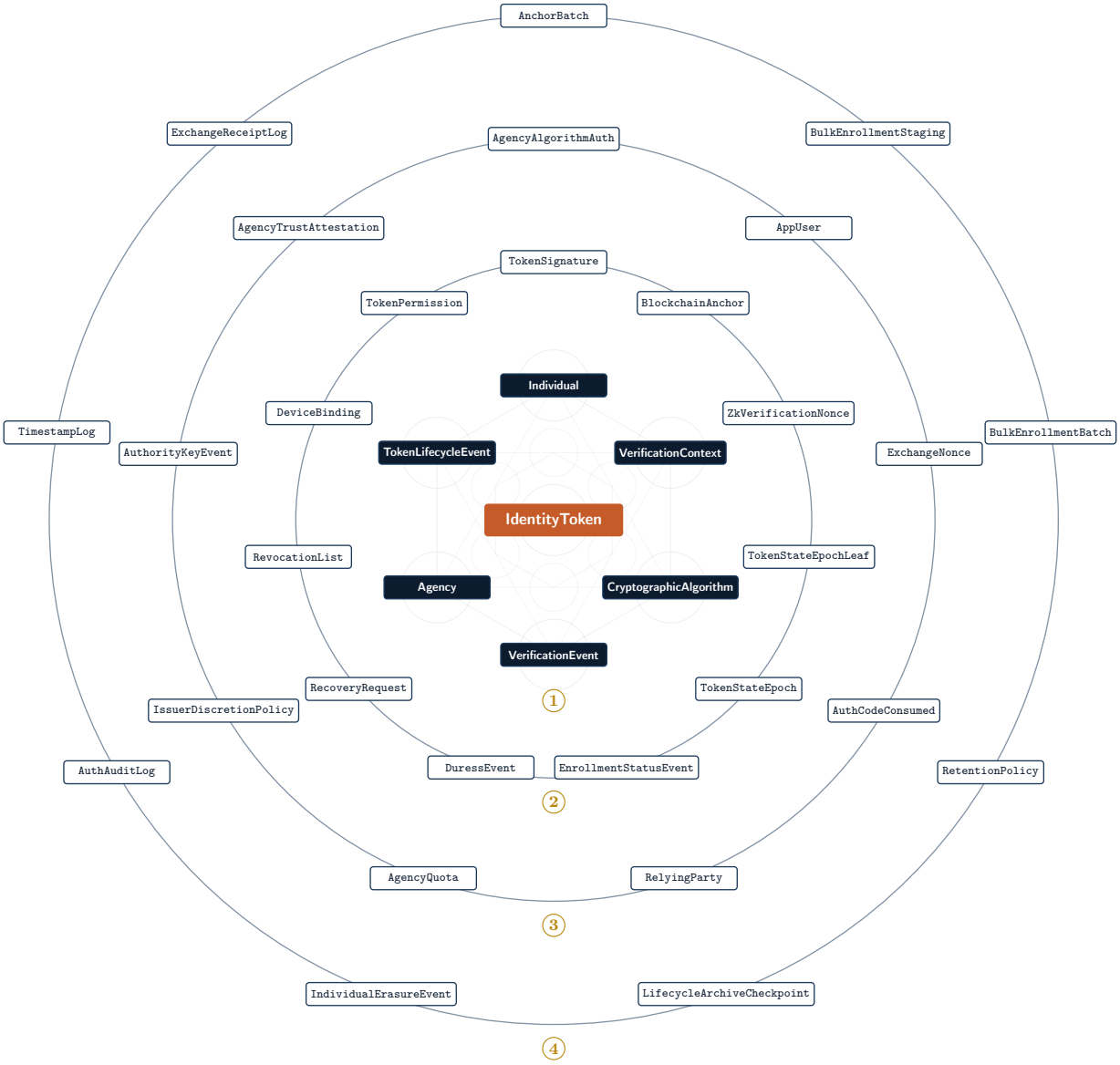


Figure 4: The 36 canonical tables of `01_schema.sql` at `v9.345`, as rings around the Version 1 nucleus. The faint lattice at the centre is the nucleus geometry of Figure 2 at the same proportions. Ring two is the credential machinery: signatures, permissions, epochs, recovery, duress. Ring three is authority and trust: who may sign under which algorithm, who trusts whom, revocation ceilings, quotas, relying parties, the replay registers. Ring four is the archives and watchtowers: append-only logs, anchors, the retention decision as data. A migrated deployment holds seven more tables (the migration registry, operator sessions and credentials, the audit-access log, and Athena's three curated tables), and four partition children.

4.3 The audit of record

Thirteen tables follow one named pattern, the audit of record: the row and its history are the same object. There is no separate event log beside the table that could be edited while the table is locked. Mutation is either forbidden outright or bounded to one field that moves one way, such as a signature's deprecation date or an attestation's revocation date. Every one of the thirteen is enforced by a trigger that raises `insufficient_privilege`, and a check fails the build if any trigger stops doing so. `RecoveryRequest` was the one instance resting on a partial unique index and procedure discipline, so a raw update from a database session was not refused; at `v9.347` its decision fields moved one way under a trigger like the other twelve, and the check now covers thirteen of thirteen. IMPLEMENTED

The corollary is that no foreign key anywhere in the schema cascades a delete. A cascade would let the lifecycle events of a deleted token vanish with it. Every reference defaults to `NO ACTION`, so a principal referenced by any audit row is effectively undeletable, and a check scans every SQL file for both cascade forms with no allowlist.

The application role, `polaris_app`, is revoked UPDATE and DELETE on the append-only tables and holds no data-definition privilege. The only path that deletes audit rows is the archive-then-purge procedure, which verifies the archive against its manifest first, refuses any cutoff inside the retention window, and appends its own append-only checkpoint row. The retention window itself is data with a floor: no policy row can express a retention shorter than a year, because a check constraint refuses it, and shortening the floor is a schema change answered for in review. The vocation refuses unbounded retention; it equally refuses retention short enough to erase the record. IMPLEMENTED

4.4 The check layer, and the checks that check the checks

Above the schema sits a flat layer of 191 plain functions, each of the form `check_*(repo_root)`, that read the repository and return a finding. They gate continuous integration: any failure fails the build. Each check is paired with a detection test that breaks a fixture in the specific way the check claims to catch and asserts that the check turns red. A check that cannot detect its own violation is treated as broken. This is the mechanism by which prose claims are turned into gates: a count stated in the README, a route that must return a header, a migration that must not cascade, a drill that CI must run. IMPLEMENTED

Layer card: the walls

What it is	PostgreSQL 16: 36 tables, 26 triggers, the stored procedures that implement every state-changing use case, the append-only audit tables, the retention policy as data.
Why it exists	A promise enforced only by the people who run the system is not a promise. The schema binds every client, every restore and every insider with a database connection short of the owner.
What it trusts	The database engine and its owner; the correctness of the constraints as written.
What it refuses	Application code, review process and operator discipline as the last line: the application never enforces a constitutional guarantee on its own.
What it can see	Everything the system stores. This is why the privacy layer bounds what is stored, not who may read it.
What it cannot see	Biometric templates, holder-side keys, the content of exchanged messages or timestamped documents: none enter the database.

If it fails	A dropped trigger, a lowered retention floor or a cascade fails the build through the checks; a database owner with a shell has larger options than any trigger, which the threat model states rather than hides.
Checked by	`check_aor_append_only_triggers`, `check_aor_privilege_boundary`, `check_no_fk_cascade`, `check_retention_engine`, the SQL self-tests, the CHECK-constraint suite.
Now or later	Now.
Inside and outside	Inside: the credential core. Outside: cryptographic evidence, because a constraint binds only the database that holds it.

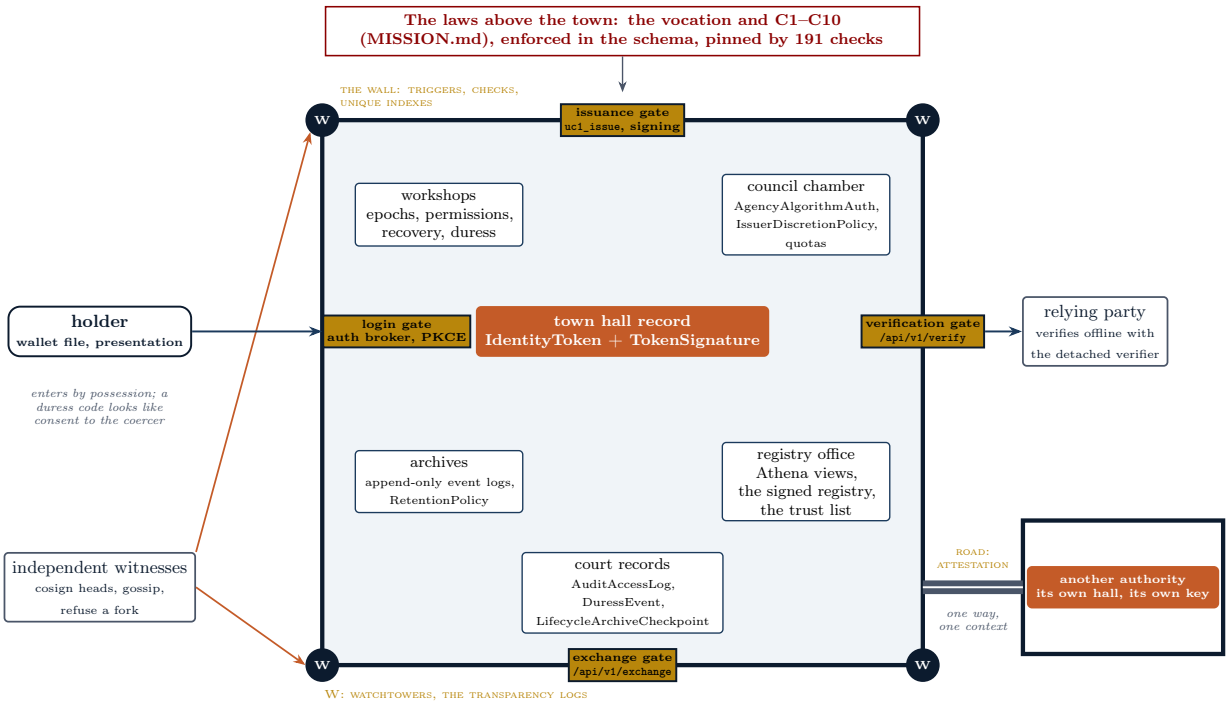


Figure 5: The walled town, with the real component beside every civic name. The register in the hall is the credential core; the wall is the schema; the four gates are the routes through which a claim enters or leaves; the watchtowers are the transparency logs and the witnesses outside the wall watch them; the road to the second town is a directional, per-context attestation; the laws above the town bind the town itself. The plan describes structure. Polaris is federated: every authority is its own town.

> *A constraint binds the database, not a copy of the claim that has left it. The moment a credential is handed to a stranger, or an authority's history is read by an auditor, nothing in the schema travels with it. The next layer makes claims carry their own proof.*

5 Authenticating truth: signatures, custody and two witnesses

5.1 What a signature is for

A digital signature is a seal that only the holder of a private key can make and that anyone with the matching public key can check. Polaris signs the SHA3-256 digest of a token value under the issuing agency's ML-DSA key and stores the signature beside the public key that made it. The claim a signature carries is narrow and permanent: *this key signed these bytes.* It says nothing about whether the token is still authorized, which is the next section's problem, and nothing about who holds the key, which is custody's problem. IMPLEMENTED

The algorithm is ML-DSA-65 (FIPS 204, security category 3) by default and ML-DSA-87 (category 5) beside it; ML-DSA-44 is refused everywhere as below the floor. No signed body hardcodes its algorithm: the custodied key decides it before signing, because `algorithm` is itself a signed field, and the registry advertises the accepted set. SLH-DSA, the hash-based alternative that does not rest on lattice assumptions, is registered in the algorithm table so that a rotation away from lattices is a row update, and no SLH-DSA signer is wired; the seed token filed under it can never be re-signed, and the posture ledger carries the gap. IMPLEMENTED for ML-DSA; PROPOSED for a hash-based signer.

5.2 Custody

The private key never sits in application code. It sits behind a custody interface with three drivers: a file (development), PKCS#11 (a hardware security module or a software token; the key is generated inside the token and never leaves it), and a cloud key-management service. Continuous integration exercises the PKCS#11 driver against a software token that implements ML-DSA, not a hardware module, and rotates a key inside it: a token signed under the old in-token key still verifies after the switch while the new key signs. One production profile makes the module the sole signer: the application refuses to boot if a file key is present or real signing is unavailable. Which driver, which module, and who holds the key are decisions the readiness ledger assigns to a deploying organisation. IMPLEMENTED for the drivers and the drill; EXTERNAL VALIDATION REQUIRED for hardware custody, since no hardware module has been used.

The placeholder deserves a paragraph, because it is where an honest system is most easily misread. Without the real-signing flag and a lattice library present, the signing module produces a deterministic SHA3-256 binding labelled `DETERMINISTIC-PLACEHOLDER-SHA3-256`, so that a developer without the library still exercises every structural invariant. It is not a signature and cannot be mistaken for one, because the label is checked. Since `v9.280` a production boot fails closed unless real signing is actually available, and the placeholder is a named development profile that warns loudly when used unnamed. The repository's own history records why: at `v9.58` the headline post-quantum claim was, at the data level, a hardcoded string, and the real signing module was an unused island until it was wired into issuance. IMPLEMENTED

5.3 Two witnesses at issuance, one at use, sampled

The two-witness principle states that any cryptographic verdict Polaris ships must be checkable by a second, independent implementation, written separately, differently represented, sharing no code, and agreeing on the verdict or abstaining explicitly. A verifier that agrees with itself has established nothing. At issuance the signature just produced is verified by two independent FIPS 204 implementations, a lattice library and an OpenSSL-backed one, and is refused persistence unless both accept it; a missing witness at issuance is a refusal, not a downgrade. IMPLEMENTED

Verification at use runs one witness, because a stored signature is already known to verify under both, and one witness detects tampering at about ten times the rate: on the reference machine roughly 7,800 verifications per second per core against roughly 740 two-witness. The fast path earns its speed by being continuously checked: a mandatory random fraction of successful checks is replayed through the second witness, a disagreement increments a counter and pages at the highest severity, and every response names which witness set ran. Two witnesses that are optional would be one witness with extra documentation. IMPLEMENTED

The same principle governs the zero-knowledge verdict (Section 8): a Python re-implementation with plain integers modulo a prime, sharing no code with the Rust prover, re-derives the membership statement and must agree, and it abstains, in the open, on the one axis it cannot model, the integrity of the proof bytes. What the principle catches is implementation

divergence. What it does not catch is a shared misreading of the specification, and the repository says so beside every two-witness claim.

5.4 Canonical signing: the same bytes on both sides

Every signed artifact in Polaris is a JSON statement with sorted keys and no whitespace, digested with SHA3-256, then signed. The application builds the statement; the detached verifier (Section 6) rebuilds it from the artifact to check the signature. The two constructions must be byte-identical or a genuine artifact fails to verify, and a test, the canonical-equivalence oracle, pins every signed type on both sides. The one bug the oracle is there to catch was found in review: an artifact whose `algorithm` field was set after signing carried a signed field the signature did not cover. IMPLEMENTED

5.5 Keys have a lifecycle

Before `v9.328` an authority had one current key and every surface said active. The key register is now an append-only table of events, registered, retired, or compromised from an instant that may predate the discovery, from which a signed trust list is published. A verifier holding a trust list decides a key's status at any instant, so a credential under a compromised issuer key is rejected independently of the issuer's own manifest, which a thief holding the key could forge, and a document signed and timestamped before the compromise stays valid while one signed after does not. The two-instance drill records a compromise on one authority and watches the other's decision flip over the wire, with no cooperation from the compromised party. Algorithm migration is a key event: register the ML-DSA-87 key, retire the ML-DSA-65 one from an instant. IMPLEMENTED

5.6 What remains classical

The token signature, the digests, the Merkle commitments, the zero-knowledge proof and the client-to-edge key exchange for modern clients are post-quantum. The two internal transport hops inside the deployment, the certificate signatures, and the operator's WebAuthn credentials remain classical, each gated on a third party (an OpenSSL version in two container images, the public certificate ecosystem, authenticator hardware) and each stated in the posture ledger with its threat and its exposure. The relying-party side already offers ML-DSA-65 first for WebAuthn so that the day an authenticator implements it, enrolment is post-quantum with no change. None of the lattice library, the proof system or the circuit has had an external cryptographic audit. EXTERNAL VALIDATION REQUIRED

Layer card: cryptographic evidence

What it is	ML-DSA signatures over SHA3-256 digests, stored with their public keys; a custody interface with file, PKCS#11 and KMS drivers; two independent verifying implementations; a canonical statement format; an append-only key register and a signed trust list.
Why it exists	A claim that leaves the database needs to carry its own proof, and a proof only one program can check is a promise.
What it trusts	FIPS 204 and 202 as standardised; the two libraries not sharing a bug; the custody backend the operator chose.
What it refuses	A verdict from a single implementation at issuance; an algorithm named in code rather than by the key; a signature stored without its key; a placeholder passing as a signature.
What it can see	The bytes it signs, which are digests: the signing path never sees a person.
What it cannot see	Whether the signed claim is still authorized (Section 7).

If it fails	A forged signature fails both witnesses; a compromised key is retired from an instant by the register and every verifier holding the trust list rejects what it signed afterwards; a library bug that one witness shares with nobody is caught by the sampled disagreement.
Checked by	`check_pqc_signing_wired`, `check_pqc_second_witness`, `check_verify_witness_sampling`, `check_real_pqc_default_boot`, `check_key_rotation_drilled`, `check_trust_lifecycle`, the canonical-equivalence oracle, the attack suite.
Now or later	Now. The hardware module and the external audit are later, and belong to a deploying organisation.
Inside and outside	Inside: the schema, which stores the signature rows. Outside: the independent verifiers that check them without the issuer.

A signature the issuer's own server checks proves nothing to a stranger, because the stranger has to trust the server. The next layer takes the verification out of the issuer's hands entirely.

6 Verification without the issuer: the detached verifier, the SDKs and the conformance contract

6.1 The detached verifier

`scripts/polaris-verify.py` verifies a credential's authenticity pack with only a standard ML-DSA library: no Polaris code, no database, no network. It is the one Polaris component whose input is chosen by an adversary, because a relying party runs it on whatever a stranger presented, and it is the component through which every later protocol artifact is checked: manifests, epoch checkpoints, revocation feeds, status bundles, status assertions, transparency heads, receipts, timestamps, signed documents, registries, trust lists, presentations. Three independent ML-DSA implementations agree on the published vectors: a lattice library, OpenSSL, and a third in the TypeScript SDK. IMPLEMENTED

The verifier is held *total* against hostile input by a metamorphic fuzzer: for every signed type it builds a genuine object, confirms acceptance, then flips bits in the signature and key, mutates and drops every signed field, substitutes adversarial values, and feeds each object to every other type's verifier, asserting a clean fail-closed rejection with no exception. The fuzzer found that the verifier crashed on several classes of malformed input, an integer where a list was expected, a hex field of the wrong type, and the fix was a totality contract that converts every would-be crash into the reject the verifier already intended. A verifier that raises pushes the fail-closed decision onto a caller who may not catch it. IMPLEMENTED

An attack suite runs beside it: executable adversaries that try to forge a signature with an attacker key, tamper a byte, present a signature against a different token, relabel the placeholder as real, or present a revoked token, against the real code. An attack succeeds when a defence fails, and the build goes red if any succeeds. A grep proves a line exists; an attack proves a defence holds. IMPLEMENTED

6.2 The SDKs and the conformance contract

A relying party installs one of two verify SDKs, a Python reference and a TypeScript implementation, each answering one narrow question about a presented credential, is it authentic and is it authoritative right now, and never returning a person's data. Both are held to a language-agnostic conformance suite: a runner drives any verifier over standard input and output through the published cases, forty-eight at `v9.345` across sixteen artifact types, including the composite federation trust decision. Passing the suite is what *conformant* means, and the normative wire specification (RFC 2119 language, twenty format strings, the exact signed-field list of every artifact) is what an implementation importing no Polaris code would build against. IMPLEMENTED

That last sentence is the honest boundary of this layer. The specification, the cases and two SDKs exist, and a check pins the specification to the signer so a documented field list cannot drift from the code. An actual implementation by an external team, from the documents alone, has not happened; the roadmap marks it as blocked on an external actor. The conformance suite is the contract. The integration is unproven. EXTERNAL VALIDATION REQUIRED

6.3 Versioning, and the frozen version 1

Protocol majors live in the format string (`polaris-registry/1`), minors are advertised by the registry and are never needed to verify, and the negotiation rule is normative: reject an unknown major, accept any minor, never emit an unadvertised version, answer an unsupported one with a specific error and the supported list. Version 1 of the protocol is frozen under checksums, with the detached verifier of `v9.317` vendored beside it, and a compatibility suite runs both directions on every push: today's verifiers must accept everything version 1 published, and the pinned older verifier must never accept what today's suite rejects. IMPLEMENTED

Layer card: independent verification

What it is	A standalone verifier, two SDKs, a conformance runner over published cases, a normative wire specification, a frozen version-1 set with a cross-version suite, a fuzzer and an attack suite.
Why it exists	A verification the issuer performs asks the relying party to trust the issuer's server. Authenticity has to be decidable by anyone, anywhere, with the network off.
What it trusts	The published issuer keys the relying party chose to trust (its anchor set); a standard ML-DSA library.
What it refuses	Any Polaris code or database; any input it cannot parse; any object presented to the wrong type's verifier; any version it was not told about.
What it can see	The artifact and the anchor set. Nothing else.
What it cannot see	Whether the credential is still authorized, unless a status artifact is presented with it.
If it fails	A wrongly accepted mutation fails the fuzzer; a defence that no longer holds fails the attack suite; a drift between specification and signer fails `check_wire_spec_matches_code`; a frozen case that stops verifying fails the compatibility suite.
Checked by	`check_detached_verifier`, `check_conformance_suite`, `check_typescript_sdk`, `check_verifier_fuzz`, `check_attacks_run`, `check_frozen_v1`.
Now or later	Now, for the software. The external implementation is later and not ours to do.
Inside and outside	Inside: the signatures. Outside: the status layer, which answers the question a signature cannot.

> *Signing a credential proves who signed it. It does not prove whether that credential is still authorized: the seal on a revoked passport is as genuine as the seal on a valid one. Polaris therefore needs status.*

7 Verification is not authorization: status, revocation and the offline trade

7.1 Two results, never one

The whole product is visible in one object. A relying party asks the issuer about a credential and receives, for a revoked token:

```
POST /api/v1/verify  ->  {
  "authentic": true,                 // the ML-DSA signature is genuine
  "currently_authoritative": false,  // but this token has been revoked
  "status": "REVOKED",
  "usable": false,                   // authentic AND authoritative -> false
  "decision": "reject"
}
```

Authenticity is permanent and cacheable. Authorization is fresh and revocable. A relying party needs both, and the API refuses to collapse them into one flag, because a signature stays genuine forever while the authority under it can be withdrawn at any moment. IMPLEMENTED

The relying-party endpoint authenticates the caller as an organisation, not a person, by an OAuth2 client-credentials grant; it accepts a possession proof rather than a sequential identifier, so it cannot be used to enumerate tokens; it returns a uniform verdict on a mismatch, so it is not an existence oracle; and it keeps no record of who verified whom. IMPLEMENTED

7.2 Fresh state, not a stale replica

The verify endpoint is routed to a read replica for throughput, and the repository's history records the mistake that split the two results permanently. Authenticity is a property of immutable material, the signed value, the signature bytes and the stored key, so a lagging replica can neither forge it nor deny it. Whether the token is usable *now* is a current-authorization question, and a revocation lands on the primary; a replica inside its staleness window could still show a just-revoked token as active. Since `v9.264` the status that decides `usable` is pinned to the primary, the response names its source and carries `as_of` and a zero staleness bound, and a caller may cache authenticity and must re-read authorization. IMPLEMENTED

7.3 Revocation

Revocation is a status transition through one sanctioned procedure that also writes to `RevocationList` in the same transaction, so the verifier-facing list can never diverge from the token's state, and a trigger refuses any other path into `REVOKED`. The list carries a canonical reason vocabulary; the audit trail carries the procedural detail and the co-signer when the issuer-discretion ceiling required one. Revocation looks forward: events recorded while a credential was valid are not invalidated, because history is not rewritten. IMPLEMENTED

7.4 Authorization with the network off

An offline verifier cannot ask the issuer. Polaris answers with a short-lived signed status assertion: the issuer's signature, under the same key that signed the credential, over the token's current status and a window. The holder fetches it by presenting the genuine credential, with no bearer and no personal data, and staples it to a presentation. A verifier accepts offline only if the credential is authentic, the assertion is authentic, bound to this credential, fresh on the verifier's own clock, no wider than the window the verifier will accept, and `ACTIVE`. IMPLEMENTED

The cost is stated as plainly as the benefit. Because verification touches no issuer, the issuer never learns it happened: offline is more private than online. And because it touches no issuer, a revoked credential's last assertion stays valid until it expires. Issuer non-observation, offline availability and revocation freshness cannot all be maximised at once; a deployment picks two, and the window is a policy number, not a law.

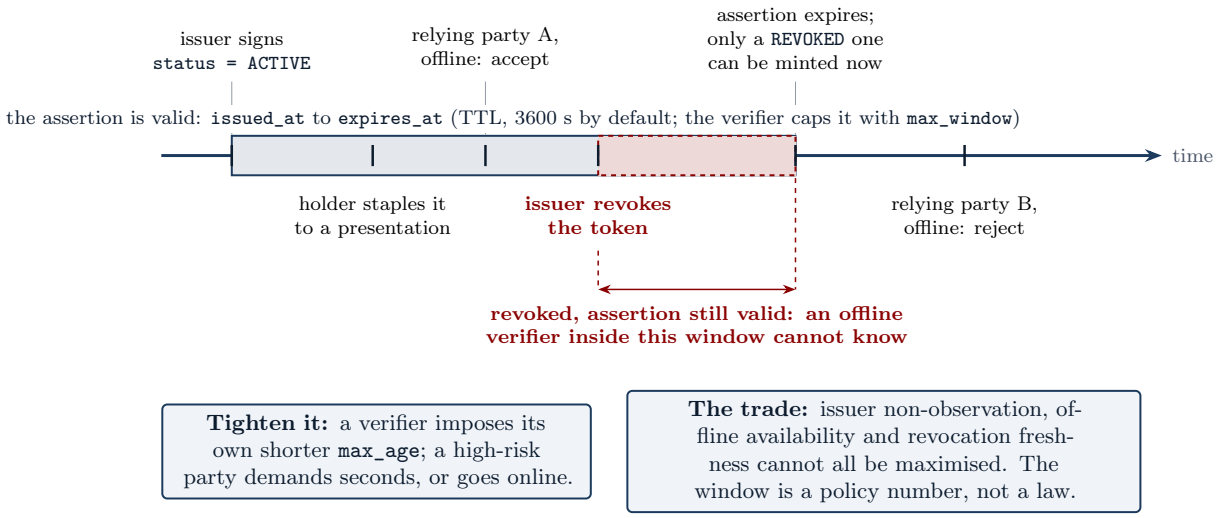


Figure 6: The offline trade, drawn as a timeline. The issuer signs a status assertion with a window; the holder staples it; a relying party verifying offline inside the window accepts. If the issuer revokes the credential inside the window, the last `ACTIVE` assertion stays valid until it expires, and an offline verifier cannot know. The window is bounded by the issuer's time-to-live (an hour by default) and by the `max_window` the verifier imposes; a high-risk relying party demands seconds or goes online.

7.5 Status across authorities

Two more signed objects carry status between authorities without a callback the issuer could log. An epoch checkpoint is an authority's commitment to the latest point on its epoch chain, the epoch number, its Merkle root and the prior epoch it extends; two checkpoints prove monotonicity, and two different roots signed at one epoch number are cryptographic proof that the authority equivocated about its own history. A revocation feed is the sorted set of revoked-credential leaves, each the SHA3-256 of a token value, derivable only by the holder; a newer feed that drops a published revocation is a caught rollback, because the underlying table is append-only. At federation scale an aggregator mirrors many authorities' feeds and checkpoints, verbatim and still under each member's own signature, into one short-lived bundle. The aggregator is untrusted for correctness: in full control of the bundle, recomputing the commitment and re-signing the envelope, it still cannot forge a member's status, because it cannot sign as the member. The bundle adds availability, not trust. IMPLEMENTED for the artifacts and drills; PROPOSED for distribution at production content-delivery scale, which the roadmap names and no deployment has.

Layer card: verification and status

What it is	The relying-party verify endpoint, the revocation list, the signed status assertion, epoch checkpoints, revocation feeds and the aggregate status bundle.
Why it exists	Authenticity is not authorization. A verifier needs to know not only that a claim was made but that it still stands, and sometimes needs to know it with the network off.
What it trusts	The issuer's signing key, for the assertion; its own clock, for freshness; the primary database, for the online verdict.
What it refuses	A replica's opinion of current status; an assertion wider than the verifier's ceiling; an aggregator's word for a member's status; a feed not bound to the issuer's key.
What it can see	The status of one credential at one instant; the set of revoked leaves.
What it cannot see	A revocation that happened after the assertion it holds was signed; the active population, which no feed publishes.

If it fails	A stale verdict is bounded by the window; a forked epoch chain or a rolled-back feed is non-repudiable evidence against the authority; a lying aggregator is caught at the member's signature.
Checked by	`check_relying_party_api`, `check_offline_verification`, `check_epoch_revocation_propagation`, `check_federation_status_bundle`, the verify-load drills under failover.
Now or later	Now, for the artifacts. Distribution at national scale is later.
Inside and outside	Inside: the verifiers. Outside: the privacy boundary, which decides what a verifier may learn while it checks.

Status proves current authority, but proving it can reveal far more than the verifier needed, and every verification is a fact about a person's day. The next layer bounds what anyone learns.

8 The privacy boundary: what a verifier is allowed to learn

Two things in Polaris use zero knowledge, one thing deliberately does not exist, and the distinction between them is the most important sentence in this section.

8.1 Three disclosure levels

Every verification event is recorded at one of three levels. `ZERO_KNOWLEDGE` proves that a valid token exists without recording which: the event's token identifier is null, and a check constraint refuses any row that says otherwise (C2). `SELECTIVE` records named attributes only. `FULL` records the identity and is logged and rate-limited. The level is decided by the server and cannot be upgraded by the client (C6): a relying party cannot turn a zero-knowledge check into a full one by asking differently, and the Atlas redacts a zero-knowledge event's location server-side. The default level is zero-knowledge. IMPLEMENTED

8.2 Issuer-side unlinkable verification records

The first use of zero knowledge is a property of the issuer's database. A default-mode verification stores no token identifier, so the verification graph, who was verified where and when, cannot be reconstructed from the database even by someone holding all of it. This is a structural claim, not a promise: the repository carries a redaction proof that the redacted field is non-derivable under a stated adversary model, and property tests exercise every read path. The claim is scoped precisely. It covers zero-knowledge-mode events. Selective and full events do carry a token identifier, because that is what those levels mean. IMPLEMENTED

8.3 The membership proof

The second use of zero knowledge is a proof object. A Merkle tree behaves like a cryptographic fingerprint of an entire set: instead of handing a verifier the whole set, one can hand it a short proof that a particular item belongs to the set with the committed fingerprint. Polaris commits, per epoch, a Merkle root over the active-token set, and a holder can produce a Plonky2 proof that its token is a leaf under that root, bound to the epoch, a context and a nonce, without revealing which leaf. The verifier sees the proof bytes and four public inputs and cannot recover the leaf; that is the proof system's property, not an API rule. The setup is transparent, with no ceremony, and the commitments are hash-based, which is why it is comfortable in a post-quantum threat model. The default tree depth of fourteen covers the schema's ten-thousand-leaf epoch cap, so the anonymity set is a full epoch. On the reference machine a proof takes about 35 ms to make and about 13 ms to check, and the verdict is two-witnessed at the statement level. IMPLEMENTED

What the proof does not do is stated with the same care. It proves membership and binding, nothing else: whether a token is active, permitted for the context and unrevoked is decided when

the epoch is composed, not inside the circuit. The nonce prevents substituting a captured proof into another context; online, a single-use nonce store refuses a replayed bundle, and offline the verifier keeps no state, so a relying party that needs single use issues a challenge. Two limitations this section carried have since been closed and are recorded here as such. The proof now carries a per-verifier nullifier, `Poseidon(secret || scope || epoch)`, so a relying party can enforce *one human, once* within its own scope while another cannot correlate its nullifiers; and the prover runs on the holder's device, against a published anonymity set, so the issuer is no longer asked which member is proving. Both rest on the epoch leaf having become a Poseidon commitment the circuit opens: while it was an opaque digest computed outside, a nullifier beside it would have proved only that the prover knew some number. IMPLEMENTED

Two bounds remain and are not closed. The nullifier does not hide the holder from the *issuer*, which derives every leaf in order to build the tree and could compute any member's nullifier in any scope; the property is between relying parties. And the circuit over the proof library has had no external cryptographic audit; the repository's soundness ledger says not to protect real identities with it as shipped. EXTERNAL VALIDATION REQUIRED

The strongest attack on the proof is not on the proof. It is correlation: matching zero-knowledge events with selective or full ones across contexts and timestamps to rebuild the graph the proof was hiding. What answers it is everything around the circuit, the redaction of the graph from every operator surface, the bounded result sets on every read path, and the threat model's accepted residual that a sufficiently resourced adversary with many data sources may still deanonymise by traffic analysis.

8.4 Bounded, not permanent

Polaris is a full-credential presentation system, and until recently the consequence was flat: the login token's subject was the credential's hash, the same value at every relying party, so two of them could join their user tables exactly and forever without either doing anything wrong. The identifier they had been handed was a global one.

That is no longer true, and the honest restatement has two halves. The subject and the presentation handle are now derived under the relying party's own scope, so each is stable where an account needs it and unrecognisable at the next verifier. Since the subject is the value a relying party *writes down*, and written-down values are the ones pooled, sold, subpoenaed and breached, this is the half that matters most in practice. IMPLEMENTED

The other half is what did not change. A full-credential presentation still shows the verifier a stable `token_value`, the issuer's signature and the holder's public key. Two relying parties who deliberately keep the raw material can still correlate. So the guarantee is about what a verifier should *store*, not about what it is *shown*, and the detached verifier says which it is giving: a plain presentation is reported as `exposed`, never as unlinkable. Withholding the material entirely needs the zero-knowledge path, where the handle is the scoped nullifier and the verifier sees no stable credential at all; that presentation is reported as `bounded`.

So cross-verifier correlation is bounded rather than permanent. Blinded presentations, one-time presentation tokens and anonymous credentials remain unbuilt, and the technologies that would remove the remaining exposure are discussed in Section 18. *Unlinkable by default* still means the issuer's zero-knowledge-mode records, and still does not mean that a verifier looking at a full credential learns nothing stable. IMPLEMENTED for the scoped derivations, with the residual exposure stated rather than closed.

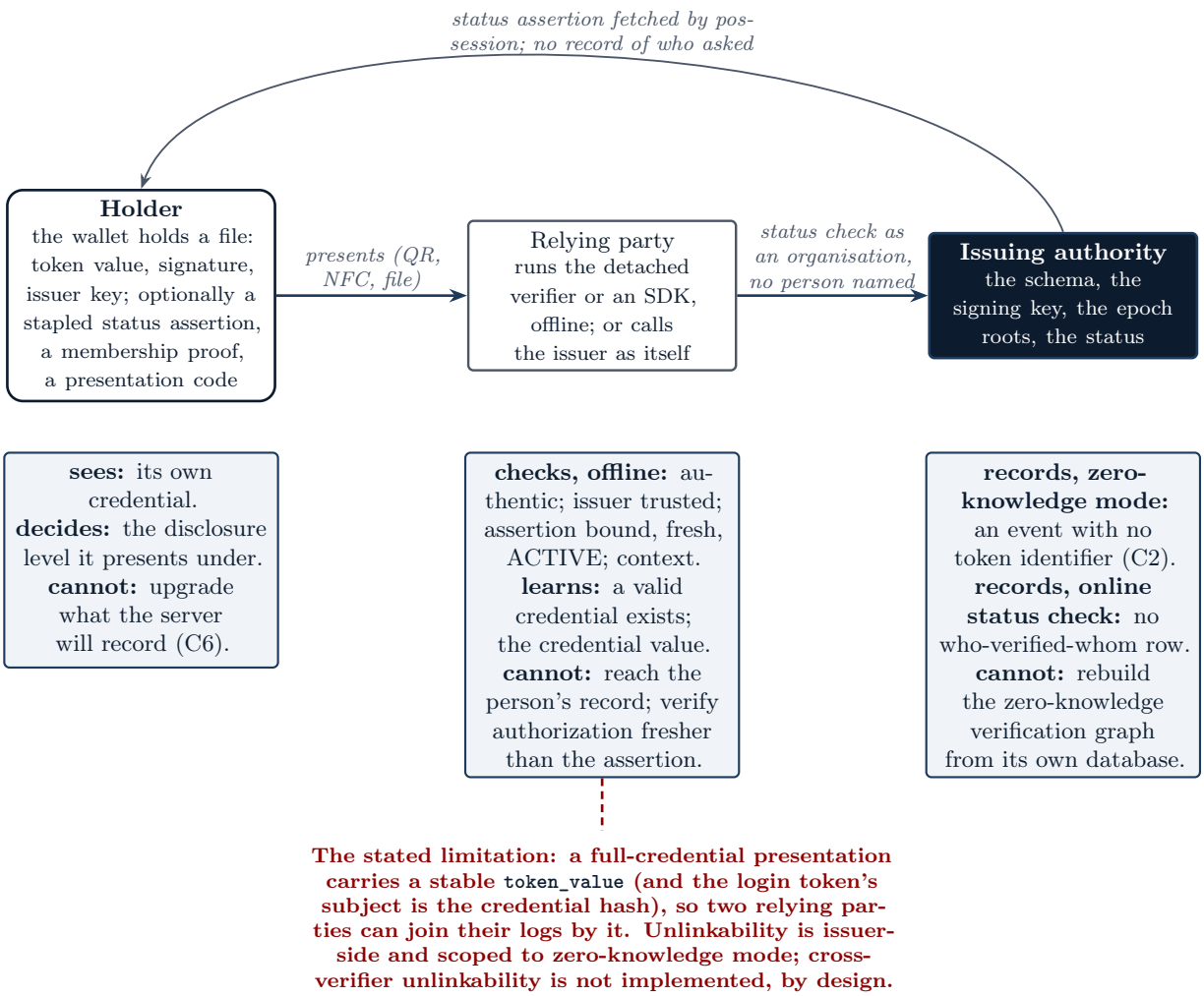


Figure 7: Holder, relying party and issuer, with what each sees and what each is prevented from seeing. The holder presents a file; the relying party decides offline with the detached verifier or an SDK and learns that a valid credential exists and its value; the issuer records, in zero-knowledge mode, an event with no token identifier, and keeps no who-verified-whom row for the online status check. The red edge is the stated limitation: the credential value is a stable handle across the verifiers it is shown to.

8.5 Bounded surfaces

The remaining privacy mechanisms bound what a surface can return rather than what is stored. Every map and API aggregate is hard-capped (C8), so the whole population cannot be extracted through the operator's atlas, and a request for six and a half million cluster bins returns at most five thousand. The single-entity investigation pages are built to refuse cross-individual link analysis and are audited: a meta-audit table records who read which audit surface, with what filter, and how many rows came back, and it is append-only. The authority-structure layer, Athena, is forbidden by five checks from referencing any natural-person table or column (Section 14). And timestamps, the timestamp authority and the exchange gateway are built to learn nothing about what they touch, which Section 12 describes. IMPLEMENTED

Layer card: the privacy boundary

What it is	Three disclosure levels with a CHECK behind the default; server-side enforcement; the redaction proof; the epoch membership proof and its second witness; bounded aggregates; an audited single-entity investigation surface.
Why it exists	A verification log is a surveillance backbone unless it is structurally unable to be one.
What it trusts	The C2 constraint; the proof library's soundness; the redaction proof's adversary model.
What it refuses	A client's claim about its own disclosure level; a token identifier on a zero-knowledge row; an unbounded read; a person node in the authority layer.
What it can see	At zero knowledge: that a valid token was verified at a time in a context. At full: the identity, logged.
What it cannot see	Which token, in zero-knowledge mode, even from the whole database.
If it fails	An identifier on a zero-knowledge row is unstorable; a cap removed fails the build; a person reference in Athena fails five checks. Cross-verifier correlation is not a failure; it is the documented boundary.
Checked by	The C2 CHECK, `check_c6_atlas_redacts_zk_location`, the redaction property suite, the ZK second-witness differential, `check_athena_no_person`.
Now or later	Now. Linkability-resistant presentation is a different system and later.
Inside and outside	Inside: status. Outside: the holders and relying parties who use the credential.

A credential bounded in what it reveals still has to be used: held, presented, accepted and logged in with. The next layer is where the credential meets software that is not the issuer's.

9 Using the credential: holders, relying parties and the login that keeps no record

9.1 The wallet is a protocol, not a product

The reference holder agent, `scripts/polaris-wallet.py`, holds a credential as a file, shows it, verifies it offline through the detached verifier, presents it, proves membership in zero knowledge against a published epoch, signs a document on the holder's behalf and logs in to a relying party. Every message it emits is specified so that any client can emit the same. Native mobile and desktop clients, card middleware and a browser bridge are product engineering that a reference implementation on notional data does not attempt; they would follow the protocol, and never lead it. IMPLEMENTED for the protocol and the reference script; PROPOSED for native clients.

A presentation is an unsigned wrapper around signed things: the issuer-signed credential, an optional stapled status assertion so that authorization is decidable with no connectivity, an optional membership proof, the context and disclosure level presented under, and an opaque presentation code. The verifier decides it offline and reports the code as present or absent without ever interpreting it, because a duress code is matched only by the issuer, out of sight; a duress presentation is byte-for-byte the same shape as a normal one. For transfer by QR or NFC the presentation is compressed and split into frames that each name the digest of the whole; a receiver takes them in any order and refuses, before parsing anything, frames naming different digests, a missing index, a conflicting duplicate, or a reassembled payload whose digest does not match. The decoder is resource-bounded: at most 9,999 frames, 512 KiB compressed and 2 MiB inflated, so a decompression bomb is refused at the limit rather than decompressed. The bounds were added after an outside review named the gap. IMPLEMENTED

9.2 The relying party

The reference relying party (`scripts/polaris-relying-party.py`) decides accept or reject for a presentation by combining offline authenticity with online status, refusing a revoked, tampered or foreign-issuer credential and staying blind to duress, and an end-to-end drill runs the whole

wallet-to-relying-party matrix under real signatures every release. A relying party registers with an issuer as an organisation with a scope that a schema constraint bounds to exactly `verify`, `authenticate`, or both, so that a fourth kind of access is a constitutional change rather than a configuration one. Identity never becomes a login product by accident. IMPLEMENTED

9.3 Logging in without a login record

The auth broker lets a relying party offer *log in with a Polaris credential* through an authorization code with PKCE. The holder presents the credential at its issuing authority's instance by possession, and the relying party exchanges the resulting code for an identity token signed by the issuing agency, which it verifies offline. Four guards keep this from becoming a login record. The token's subject is the credential's hash, never a person identifier, and it is stable per credential and therefore correlatable across relying parties, which the repository documents as a permanent property rather than pretending otherwise. The token carries no claim beyond the vocabulary: context, disclosure level, the assurance reached, enrolment status, instants. The authorize route writes nothing, and the token endpoint's only write is the code's hash into a register that holds no subject and no relying party. And duress is served identically. IMPLEMENTED

An outside review at `v9.331` found that the relying party's demands, a required zero-knowledge step-up or a required enrolment status, arrived only in the holder-side request, so nothing bound the authority to what the relying party actually required. Since `v9.336` the policy is registered on the relying party's row and applied first: a request may add a requirement and never remove one, and a context other than the registered one is refused. The same review noted that the authorization code was signed but readable; it is now encrypted, because before this flow is ever used through a browser rather than the wallet, an opaque code is the floor. IMPLEMENTED

9.4 What is deliberately not built

There is no session at the authority, no consent screen, no discovery document, and no browser redirect choreography: a relying party integrates from the wire specification and the registry. A WebAuthn browser bridge waits on a holder-side key that the issuer-centric model does not have; the holder holds a credential, not a key pair, which is why document signing is notarial and login is by possession. No relying party outside the repository has integrated. EXTERNAL VALIDATION REQUIRED

Layer card: use

What it is	The wallet protocol and reference script, presentations and QR framing, the reference relying party, relying-party registration with a bounded scope, and the auth broker.
Why it exists	A credential nobody can hold, present or accept is a database row. This layer is where the credential meets software the issuer does not run.
What it trusts	The signed objects inside a presentation; the issuer's key for the identity token; the relying party's registered policy.
What it refuses	To interpret the presentation code; to write a login record; to accept a scope outside the CHECK; to let a holder's request weaken the relying party's registered demand; to decompress an unbounded payload.
What it can see	A relying party sees the credential value and the verdicts. The authority sees a possession proof and consumes a code hash.
What it cannot see	The authority cannot see who logged in where; the relying party cannot see the person's record.

If it fails	A replayed code hits the consumed register; a wrong presentation, an unmet step-up, a verify-only bearer at the broker, a stale or tampered frame are each refused in the drills.
Checked by	`check_holder_wallet`, `check_wallet_presentation`, `check_holder_verifier_flow`, `check_auth_broker`, `check_relying_party_api`, `check_qr_resource_bounds`.
Now or later	Now, as a protocol. Native clients and a real relying party are later.
Inside and outside	Inside: the privacy boundary. Outside: federation, because a relying party rarely trusts only one authority.

A relying party that trusts one authority is easy. A society has many authorities, none of which should be able to speak for the others, and none of which should hold every person's record. The next layer is how independent authorities recognise one another without collapsing into one.

10 Federation: independent authorities that recognise one another

10.1 Why there is no centre

Two shapes were possible. A central instance holds every person's identity and every agency's tokens under one trust root, with agencies as tenants. A federated shape gives each issuing authority its own instance and its own signing key, records cross-authority trust explicitly, and lets a relying party verify against published keys with no central service in the path. The decision record chooses the second and states that it is not a free choice: the constitution names the federation trust graph as load-bearing, says no agency holds a monopoly, and refuses population-scale aggregation structurally. A central instance is a monopoly by construction and a single store to seize, subpoena or mine. The federated shape is deliberately the more expensive one; its cost is the inter-authority protocol, and the cost is accepted. IMPLEMENTED

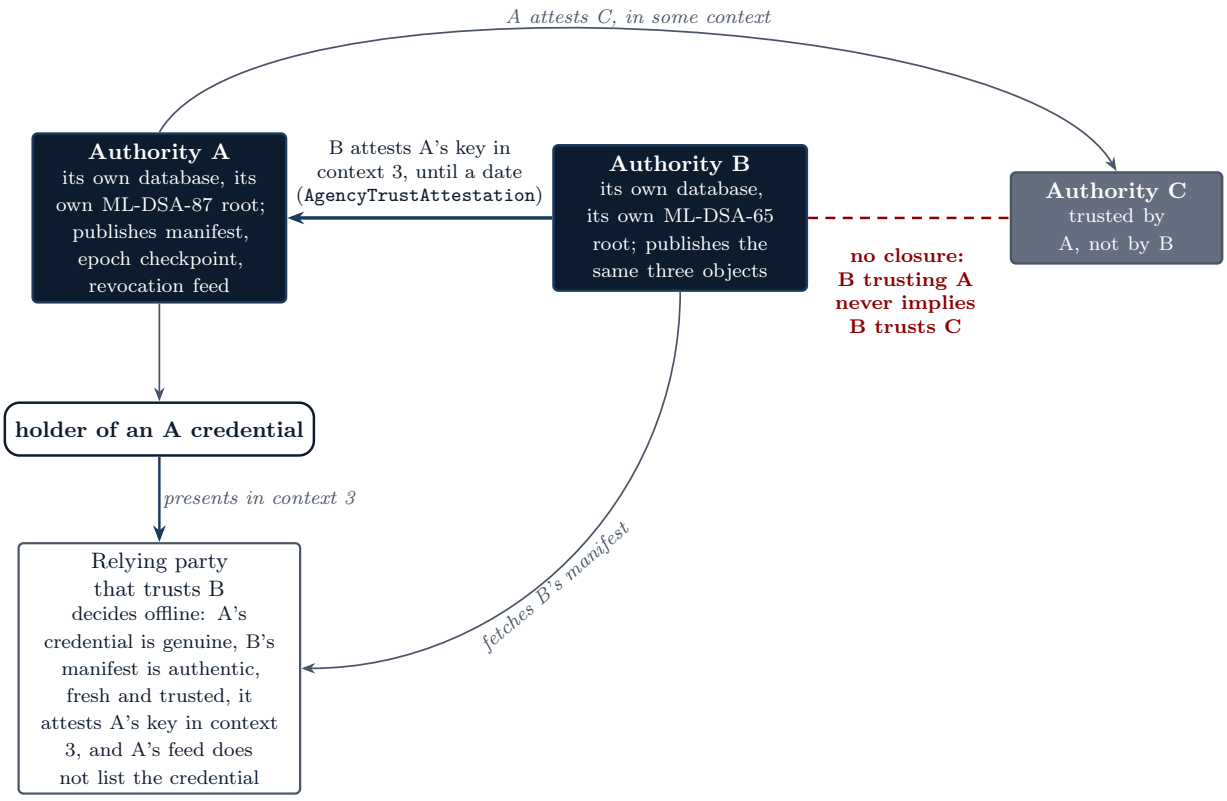
10.2 Trust is a row, not a chain

An attestation is one row of `AgencyTrustAttestation`: *verifying agency V accepts issuing agency I in context C, until a date.* Verification across agencies succeeds if and only if exactly one active row matches the verifier, the issuer and the context. The lookup is a single non-recursive query. It never computes a closure, so V trusting I never implies V trusts what I trusts; cycles cannot arise because nothing walks the graph; and an operator who wants multi-hop trust declares every hop, so the audit trail holds every decision rather than the inputs to an inference. Self-attestation is refused by a check constraint. Revocation sets a date and a reason together, once, and looks forward: events recorded while the edge was active are not invalidated. IMPLEMENTED

One limit is named rather than glossed. The row records which Polaris operator created the attestation; it is not a cryptographic signature from the attesting agency. Agency-level signing of attestations is a migration the schema is shaped for and a key-management problem the readiness ledger assigns to a deploying organisation. EXTERNAL VALIDATION REQUIRED

10.3 The three signed objects an authority publishes

A relying party needs three things from a foreign authority, and all three are signed views over append-only tables, published at short-lived endpoints and consumed offline by the detached verifier. The federation manifest carries the authority's own anchors, every key it has ever used with its real status, and the attestations it has made, each with the attested key. The epoch checkpoint is its commitment to the latest point on its epoch chain. The revocation feed is the set of revoked leaves. A manifest must be self-consistent, signed by one of the active anchors it declares, so a stranger's key cannot sign one; authentic, under two witnesses; fresh, within a



In CI this runs as two independent instances over HTTP, each with its own database and real key, under a mixed pair of parameter sets. Two instances of one code base, run by one author, prove the protocol; they do not prove institutional independence.

Figure 8: Two authorities and one directional attestation. B attests A's key in one context; a relying party that trusts B accepts A's credential in that context, offline, from B's manifest and A's revocation feed. A also trusts C, and that gives B nothing: trust is never a closure. In CI this runs as two independent instances over HTTP under a mixed pair of parameter sets.

window the consumer caps; and trusted, from an authority whose anchor the relying party chose. The cross-authority decision then reads: accept if and only if the credential's signature is genuine, the trusted manifest attests to that signing key in the presented context, and the issuer's feed, fail-closed, does not list the credential. IMPLEMENTED

A holder can also prove membership in a foreign authority's epoch in zero knowledge. The proof already carries the epoch root as a public input; the root already travels the federation signed in the checkpoint. The decision composes the two: trust the foreign checkpoint through the graph to obtain a trusted root, require the proof to bind to that root, epoch and context, then check the proof by shelling to the prover binary as a local subprocess, with no network. If the binary is absent the verifier abstains, which is never a false accept. The verdict names an authority and a decision, never a token. IMPLEMENTED

10.4 Two instances, and what they prove

A dedicated CI job boots two independent instances as two authorities, each with its own database and its own real ML-DSA root, talking only over HTTP, and drives the whole matrix: cross-verification from a fetched manifest, attestation revocation flipping the decision, epoch checkpoints and revocation feeds fetched and verified over the wire, a key compromise recorded on one side reversing the other's acceptance, the exchange gateway, the timestamp service read out of a

fetched registry, the broker's login, document signing through the wallet, and a mixed pair of parameter sets (ML-DSA-87 on one side, ML-DSA-65 on the other). It is red on any wrong decision. IMPLEMENTED

What the job proves is that the protocol works between two instances of this code base run by one author on one machine. It does not prove institutional independence, and it does not prove that an independent implementation, written by someone else from the specification, interoperates. The first requires deployment by institutions that are not the author. The second requires an external team, and has not happened. The repository draws this line itself: protocol independence, independently operated institutions and independent third-party implementations are three different claims, and only the first is demonstrated. EXTERNAL VALIDATION REQUIRED

Layer card: federation

What it is	Per-authority roots, directional per-context attestations, the signed manifest, epoch checkpoint and revocation feed, the aggregate status bundle, cross-authority zero-knowledge membership, and the two-instance drill.
Why it exists	No agency may hold a monopoly, and no single store of every person may exist; yet a relying party must be able to accept a credential issued elsewhere.
What it trusts	The anchors the relying party itself chose; the append-only tables the signed objects are views over.
What it refuses	Transitive trust; a central service in the verification path; a manifest not signed by its own declared anchor; an aggregator's word for a member's status; a callback the issuer could log.
What it can see	Published trust data: keys, attestations, epoch roots, revoked leaves. No person.
What it cannot see	The active population of any authority; who verified whom.
If it fails	A forged manifest fails self-consistency; a forked epoch chain or a rolled-back feed is non-repudiable evidence; a compromised issuer key is retired from an instant by a trust list the issuer does not control.
Checked by	`check_federation_topology`, `check_inter_authority_protocol`, `check_epoch_revocation_propagation`, `check_cross_authority_zk`, `check_federation_two_instances`, `check_exchange_trust_directional`.
Now or later	Now, as a protocol between instances. Independent institutions and implementations are later, and are not ours to supply.
Inside and outside	Inside: use. Outside: transparency, because a signed history can still be told differently to different people.

> *Signed federation objects prove what an authority said. A dishonest authority can still say different things to different observers, publishing one epoch chain to an auditor and another to a relying party. Polaris therefore needs transparency: a way for observers to prove that history only ever grew, and to compare notes.*

11 Watching the watchers: logs, witnesses and the split view

11.1 The gap after anchoring

Polaris periodically commits a batch of audit rows to a Merkle root and records the root in an append-only table, so that an outside verifier can prove that the history under a root has not been revised since. That leaves one gap. A single root proves the rows under it; it does not prove that the operator did not later discard a root and publish a different one in its place. A transparency log closes the gap by committing to the whole sequence of roots and letting anyone prove that the sequence only ever grew. IMPLEMENTED

11.2 The log

The log's entries are the anchor roots in append order; nothing new is stored, because the underlying table is already append-only. Over those entries the application builds a Merkle tree in the style of Certificate Transparency (RFC 6962), with SHA3-256 and distinct prefixes for leaves and

interior nodes so that one can never be presented as the other. It publishes a signed tree head, the log's commitment to its entire history at a size; a consistency proof that the size-m tree is a prefix of the size-n tree, which exists only if the first m entries were never reordered, rewritten or dropped; an inclusion proof for any entry; and the entries themselves, so that a monitor can replicate the log and recompute everything. The same machinery publishes two more logs: the hashes of minted exchange receipts, and the hashes of anchored timestamps. There is no personal data in any of it. IMPLEMENTED

11.3 The monitor, and what it cannot see

A monitor is a standalone program anyone runs. It keeps one piece of state, the last head it accepted; each cycle it fetches the current head, verifies the signature against a key it was given out of band, fetches a consistency proof from its last head to the new one, and either records the new head or alerts on a rewritten entry, a fork, a shrunken tree, a head signed by the wrong key, or a timestamp that went backwards. What it proves is consistency: the operator cannot rewrite history without every monitor that saw the old head detecting it. What it cannot prove on its own is that two monitors were shown the same head at the same size. A log can show one observer a private fork of history. IMPLEMENTED

11.4 Witnesses, gossip and the equivocation proof

A witness is a monitor that does two more things. It cosigns a head with its own key, only when the head is an append-only extension of the last one it cosigned, and it gossips the log-signed head it saw into a shared pool. Two uses follow, both verified offline. A witnessed checkpoint is a head carrying cosignatures from at least K distinct trusted witnesses, so a split view needs K independent witnesses to equivocate, not just the log. An equivocation proof is two heads for one log, both validly signed by the log's key, at the same size with different roots; the log signed both, so it cannot repudiate the lie, and two gossiping witnesses produce the proof the moment they compare notes. A witness that is itself shown a second head at a size it already cosigned refuses and writes the proof. IMPLEMENTED

An external ledger completes the picture: the log publishes each head into an independent append-only ledger and receives a receipt, the ledger's own signed head plus an inclusion proof, so the log cannot use a head it has not publicly committed and the ledger cannot later drop it. A ledger is itself an append-only log, so this reuses the same machinery and is monitored the same way. The file-backed ledger is implemented and tested; the chain-backed drivers are declared and wait on their interfaces. IMPLEMENTED for the file backend; PROPOSED for a chain.

11.5 Who runs the witnesses

The strength of this defence in practice is exactly the independence of whoever runs the witnesses. What is built is the protocol: the cosignature format, the threshold rule, the equivocation proof, and a witness daemon anyone can run, all drilled under real signatures every release with three witnesses catching a fork two ways. What cannot exist until deployment is a set of institutionally independent witnesses, a bank, a civil-society group, another authority, actually running it. Today the witnesses are three processes in a drill. The repository states that this is not a software deficiency but the absence of a deployment, and this paper repeats it, because a transparency layer with witnesses run by the party it watches is not one. EXTERNAL VALIDATION REQUIRED

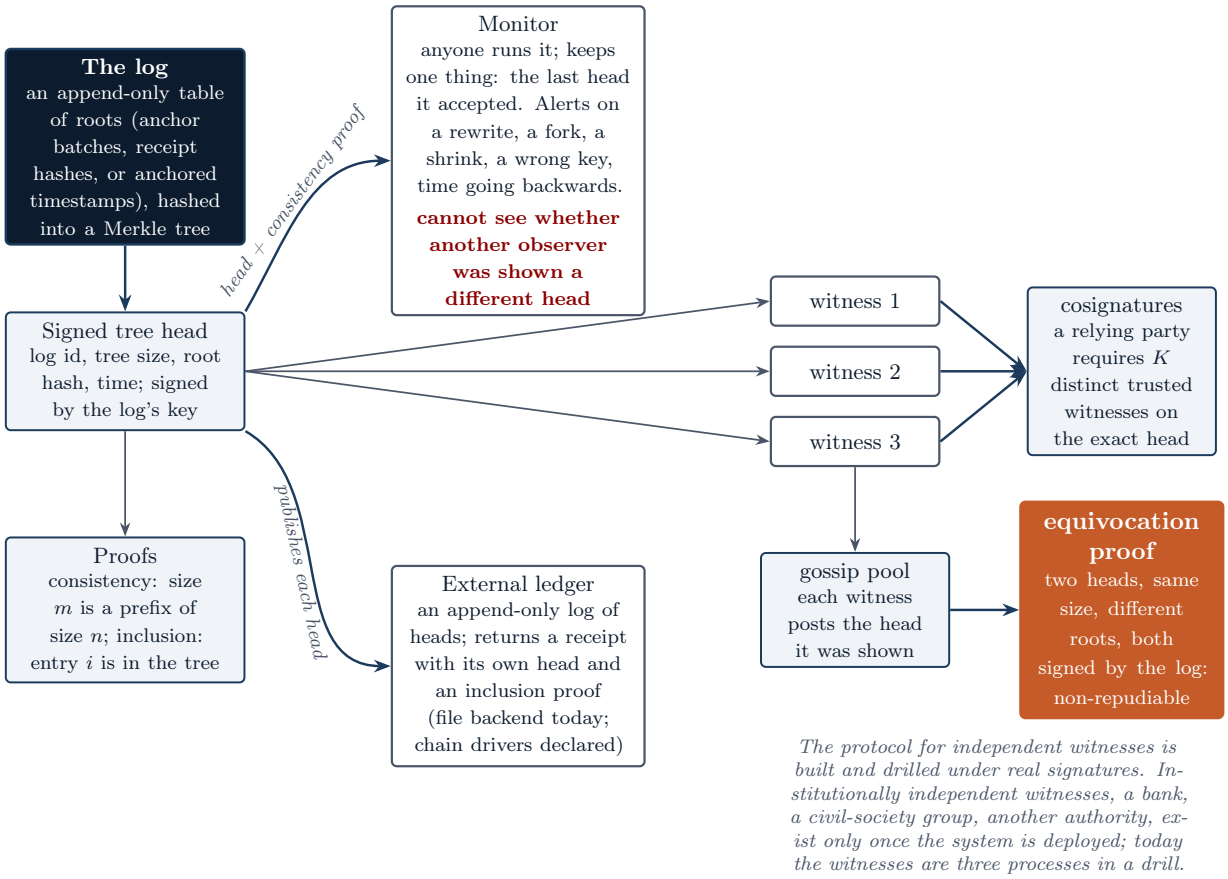


Figure 9: The log, the monitor, the witnesses, the gossip pool and the equivocation proof. The log publishes a signed head and proofs. A monitor proves the log only appended. Witnesses cosign heads they found consistent, a relying party requires a threshold of them on the exact head, and witnesses gossip the heads they were shown; two log-signed heads of the same size with different roots are a non-repudiable proof that the log lied. Heads can also be published into an external append-only ledger that returns a receipt.

Layer card: transparency

What it is	Three RFC-6962-style logs over append-only tables; signed heads; consistency and inclusion proofs; a monitor; witnesses with cosignatures, a threshold and gossip; the equivocation proof; external-ledger publication with receipts.
Why it exists	A signed object proves what was said; only a watched sequence proves that what was said was not later unsaid, and only compared notes prove it was said the same way to everyone.
What it trusts	The log's key for the head; the witnesses' keys and their independence, which is a deployment fact.
What it refuses	A head with no consistency proof from the last one; a head not cosigned by the threshold; a receipt the ledger did not record; a second head at a size already seen.
What it can see	Roots and hashes. Never a person, never a payload.
What it cannot see	On its own, a split view: that is what the witnesses are for.
If it fails	A rewrite fails every monitor; a fork is a non-repudiable proof; a dishonest witness is one of K; a dropped published head fails the ledger's own consistency.
Checked by	`check_transparency_log`, `check_transparency_gossip`, `check_transparency_publication`, `check_timestamp_transparency`, the canonical-equivalence oracle for the signed head.
Now or later	Now, as a protocol with a drill. Independent witnesses are a deployment, later.
Inside and outside	Inside: federation. Outside: the institutions that exchange, sign and timestamp under this watch.

With authority, status and history checkable by strangers, institutions can build on the credential: exchange requests, sign documents, attach time. The danger is that an exchange fabric becomes the very message log the vocation forbids. The next layer is built as its inversion.

12 Institutions: exchange, time and signatures without a message log

12.1 The inversion

A conventional exchange fabric between institutions makes an exchange provable by logging the exchanged messages: a mediating node signs and timestamps the traffic so that a receipt can later be shown to a third party. The evidence is real, and so is its cost: a store of who asked what about whom, which is exactly the population-scale record the vocation forbids. The Polaris exchange fabric is the inversion. It proves the same three things a message log proves, that an exchange occurred, between these parties, and was authorized, while retaining none of the payload, by committing to the request and the response by hash and never by content. Every primitive in it, signatures, the detached verifier, the transparency logs, the trust graph, already existed; the fabric composes them. IMPLEMENTED

12.2 The registry: discovery, not configuration

A consumer learns what an instance offers and trusts from one signed artifact: the protocol formats and versions it speaks and its accepted algorithms, each service's kind, path template and authentication, the transparency logs, every federated authority with its key register, the contexts and the proof each requires, the in-context trust graph, and the relying parties by name and scope only. Every fact is a view over the authority layer; the registry adds no truth of its own. It must be signed by the active key it lists for its own publisher, so an impostor cannot publish one in an authority's name, and it is non-transitive: it describes its publisher's instance, never another's. The two-instance drill reads a service's path out of a fetched registry and calls it. No token, no holder, no verification record appears: a map of the institutions, never of the people. IMPLEMENTED

12.3 The gateway and the receipt

The receipt commits to the requester's key, the responder, the context, the two body hashes, which attestation authorized the requester, and the instant. The mint endpoint accepts only hashes, so the application cannot retain what it is never given. A third party verifies offline that the receipt is authentic and that the requester was authorized in that context, given the manifests it trusts. A party that holds a body can confirm the hash binds; a party that does not still holds the responder's signed account. One correction from the outside review belongs here: a receipt alone cannot prove the requester took part, because a dishonest responder can sign any receipt it likes; the requester's own signed envelope beside it can, and the wire specification names the pair as the evidence. The set of receipts is itself a transparency log, so an institution cannot later deny a receipt existed and anyone can see how many it minted, while each receipt stays in the hands of its parties. IMPLEMENTED

Authorization at the gateway is directional. Since `v9.333` the responding agency itself must attest the requester's key in the envelope's context; an attestation by any other agency on the instance authorizes nothing there. Before that fix the gateway accepted an attestation from anyone on the instance, which was the transitive trust the federation layer refuses, reappearing one layer up. IMPLEMENTED

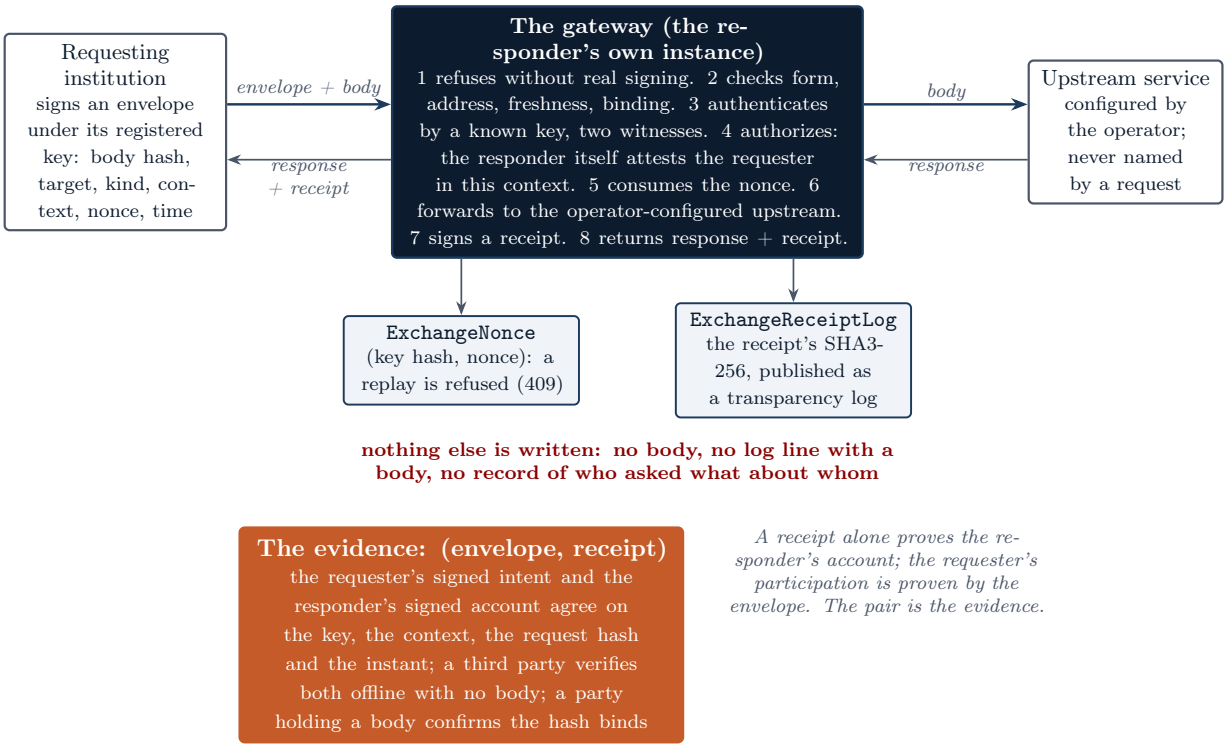


Figure 10: An institutional exchange through the gateway. The requester signs an envelope under its registered key; the responder's own instance authenticates it by a key it already knows, authorizes it through its own attestation in the envelope's context before anything leaves the process, consumes the nonce in an append-only replay register, forwards the body only to an upstream the operator configured, and returns the response with a signed receipt. Nothing but the receipt's hash and the consumed nonce is written. The evidence of the exchange is the pair of envelope and receipt, verifiable offline by a third party with no body.

12.4 Time that learns nothing

The timestamp authority binds an arbitrary SHA3-256 digest to an instant under an authority's registered key. It accepts a digest, never content, and keeps no per-request record, because a timestamp authority that logs every request is a store of who timestamped what and when. The evidence is the signed statement in the requester's hands. Since `v9.341` a caller may choose to anchor: the timestamp's own hash joins an append-only, witnessed log and the inclusion evidence comes back stapled, so that a timestamp backdated under a stolen authority key is one absent from every witnessed head of its claimed era. What the authority then retains is one digest and one instant, a record of volume and timing, never of content, and the default request still leaves nothing. IMPLEMENTED

12.5 Signing a document so that it outlives the key

A holder in Polaris holds a credential, not a signing key, so the model is notarial: the issuing authority signs on behalf of a holder who has just proved possession, exactly as it asserts status for one, and records who by hash. The authority's signature says that a holder of this credential, active at this instant, asked it to sign this digest, and the embedded evidence lets anyone check that claim later. IMPLEMENTED

Long-term validation is where the outside review and the repository's own drills sharpened the claim. An authentic timestamp is not a trusted one, since anyone can sign one, and a signer's own timestamp is backdatable by whoever holds the key. So the strong claim requires the timestamp

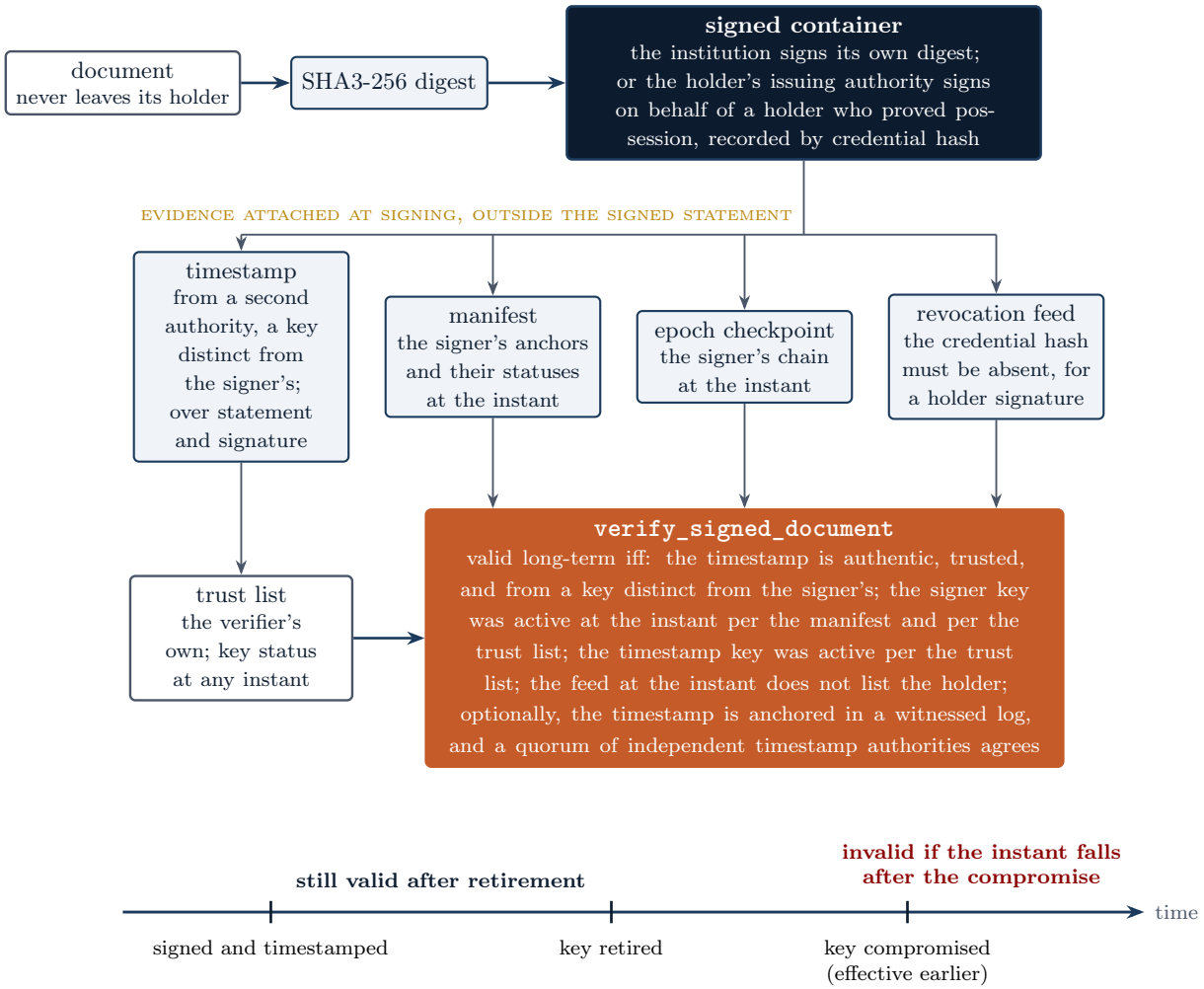


Figure 11: A signed document and its long-term evidence. The document never leaves its holder; its digest is signed by an institution, or by the holder's issuing authority on the holder's behalf after a possession proof, recorded by credential hash. Evidence fixed at the instant of signing, a timestamp from a distinct key, the signer's manifest, checkpoint and feed, together with the verifier's own trust list, lets a verifier decide validity at that instant long after the signing key has been retired. A signature whose instant falls after a compromise is refused even though its embedded manifest says the key was active.

to be trusted by the verifier, from a key distinct from the signer's, active at the instant per the trust list, and, if the verifier asks, anchored in a witnessed log or corroborated by a quorum of independent authorities. A verifier given no timestamp anchors reports the facts and claims nothing. IMPLEMENTED. Anchor verification, once available only in the detached verifier, is in both SDKs since `v9.346`, so an implementation that imports no Polaris code can decide long-term validity.

Layer card: institutions

What it is	The signed registry, the exchange gateway, the exchange receipt and its log, the timestamp authority with optional anchoring, document signing with long-term validation, the trust list.
Why it exists	Real software ecosystems need discovery, mediated requests, signatures and time. Each of those is conventionally built on a log of content, and the vocation forbids that log.
What it trusts	Registered institutional keys; the in-context trust graph; the verifier's own timestamp anchors and trust list.
What it refuses	A body at the mint endpoint; a URL named by a request; a replayed nonce; an attestation not made by the responder itself; a timestamp from the signer's own key as the strong evidence; a placeholder signature as authentication.
What it can see	Digests, keys, contexts, instants.
What it cannot see	The exchanged messages, the timestamped content, the signed document: none are ever given to it.
If it fails	A forged receipt fails the responder's signature; a dishonest responder's receipt is contradicted by the missing envelope; a stolen timestamp key is caught by the witnessed anchor or the quorum; a compromised signer key invalidates only what it signed after the instant.
Checked by	`check_registry`, `check_exchange_gateway`, `check_exchange_receipt`, `check_timestamp_authority`, `check_document_signing`, `check_ltv_timestamp_trust`, `check_broker_policy_bound`.
Now or later	Now, between two instances. Real institutions are later.
Inside and outside	Inside: transparency, which watches the receipt and timestamp logs. Outside: the operations that keep all of it running.

Every layer so far is a claim about correctness. None of it matters if the system falls over under load, loses a database, or cannot be deployed by anyone but its author. The next layer is survival.

13 Surviving reality: the stack, failure, load and the standing witness

13.1 The stack

The production topology is five services: a self-built TLS edge that negotiates a hybrid post-quantum key exchange with modern clients, the application under gunicorn, a connection pooler, PostgreSQL with continuous archiving, and Redis for the shared rate limiter. Every service runs as a non-root user with every capability dropped, third-party images are pinned by digest, and every production container drops the test framework. Four deployment paths share one secrets discipline, one backup path and one threat model: a laptop launcher, a single-host compose profile with a blue-green rolling deploy, a scripted Linux install under systemd exercised on two distributions, and a Helm reference profile with default-deny network policies and the restricted pod security standard, booted in CI on a one-node cluster. IMPLEMENTED

13.2 Failure, drilled and measured

The repository's rule is to exercise rather than read: ten of ten defects found in one sweep were invisible to reading. Every operational claim below is a drill that runs in CI and commits a number.

High availability. The database runs under Patroni with a leader lease in a three-member etcd and role-based routing. Under a live write stream the drill loses the leader (the replica promoted after the 20-second lease, in-flight queries failing fast and retried, none lost), cuts the leader off from the lease store (it stood down in 7 seconds), performs a planned switchover (3.3 seconds, no insert failed), and crashes an etcd member (a 0.3-second stall).

The Kubernetes profile runs the same members with the cluster's API as the lease store and deletes the leader pod. IMPLEMENTED

Read replicas. Read-only surfaces route to a streaming replica under an explicit staleness contract, at most ten seconds behind by default, falling back to the primary beyond it, with every routed response naming its source and lag. Correctness-critical reads stay on the primary, and the authorization half of a verification verdict is pinned there (Section 7). IMPLEMENTED

Partitioning. The four append-only event tables are monthly range-partitioned, and a drill proves that the append-only trigger holds across a partition, an attach and a detach, because the archives' carve-out must not become a rewrite path. A benchmark found that the Atlas roll-ups defeated partition pruning under the generic plan and scanned every month of history; the fix is pinned by a check. IMPLEMENTED

Disaster recovery. Backups are encrypted and a restore fails closed without the key. A monthly drill kills the primary, restores it from the archive and commits the measured numbers against the targets of 300 seconds of data loss and four hours to recover: the last recorded rows show 36 to 42 seconds of loss and under five seconds to service, on the drill's ephemeral host, not production hardware. IMPLEMENTED

Chaos. A weekly drill boots the blue-green stack under traffic, kills one colour with zero dropped requests, stops both until the outage page reaches a real alert manager and webhook, kills Redis and PostgreSQL, partitions the pooler, and commits every recovery time against a ceiling. Findings become checks. IMPLEMENTED

Rolling deploys. A deploy under traffic drops zero requests; edge and database recreation are window operations measured on every push against ceilings (0.3 seconds for the edge on a single host, 0.6 seconds for a database crash), and the readiness ledger carries the remaining edge half openly. IMPLEMENTED

13.3 Load: what was measured and what was projected

Three kinds of number appear in the repository, and the paper keeps them apart. *Measured* numbers come from a named machine, an Apple Silicon laptop with eight cores, and carry a version stamp. *Simulated* numbers come from the national simulation harness, a synthetic country driven through the real procedures at a stated scale factor. *Projected* numbers are arithmetic over measured ones and are labelled as such by the repository itself. No production cluster has been measured.

Table 12: Performance numbers at the versions that measured them. Every row is single-node on the reference laptop unless marked as a projection. The event-ingestion rate and the cryptographic verification rate differ by about 35 times, a gap the repository's own history once hid (Section 15).

Quantity	Number	Kind	Source
ML-DSA-65 sign, one core	about 2,110 per second	measured	dyno, `v9.278`
ML-DSA-65 verify, single witness, one core	about 7,840 per second	measured	dyno, `v9.278`
ML-DSA-65 verify, two witnesses, one core	about 740 per second	measured	dyno, `v9.278`
Membership proof, depth 14	about 35 ms to prove, 13 ms to verify, 76 KB	measured	dyno and the soundness ledger
Issuance through the application	40 per second sustained, p95 28 ms	measured	baseline, `v9.191`
Verification event through the application	80 per second sustained, p95 18.5 ms	measured	baseline, `v9.191`

continued on the next page

Quantity	Number	Kind	Source
Enrolment with real signing, bulk pipeline	about 372 tokens per second	simulated, 1:10,000	national simulation, `v9.257`
Verification-event ingestion (audit writes, no signature check)	about 25,970 per second	simulated, 1:10,000	national simulation
Atlas at ten million events: a street block; the whole world; the whole world from a roll-up	2.6 ms; 2.9 s; 0.04 ms	measured, `v9.150`	scaling ledger
Single-witness verification across an eight-core node	about 62,800 per second	projection	benchmark arithmetic

The roadmap's planning targets (ten million persons for a state; 350 million, five thousand sustained and fifty thousand peak verifications per second nationally) are targets to be validated, stated in the repository as such and not asserted. A national workload has been simulated at a scale factor and driven through the real constraints; it has not been run. EXPERIMENTAL

13.4 Abuse, secrets and supply chain

Per-agency quotas on issuance, revocation and verification are opt-in rows enforced by a trigger on every write path, exact under concurrency by an advisory lock, refused loudly on every surface, and keyed by agency identifiers, never by a person. Velocity alerts compare each agency to its own trailing week. The per-IP rate limiter shares one Redis bucket across workers. Operators authenticate with WebAuthn as a second factor after an enrolment deadline, sessions are registered server-side with per-role caps and origin checks, and a per-role network policy applies. Secrets are file-mounted, sealed and rotated through one lifecycle; the application refuses to boot in production on a default key. Every release ships bills of materials with signed provenance, and CVE scans of the dependency surface and of every self-built image gate the build. IMPLEMENTED

13.5 The standing witness

Eighteen CI jobs run on every push, and most carry a comment naming the past failure they exist to prevent. They build and boot the images; boot the five-service stack end to end and assert health through the TLS edge; round-trip an encrypted backup and an off-site copy; kill and restore the primary; roll a deploy under traffic; install the Linux profile on two distributions; boot the Kubernetes profile; page a duress alert through a real alert manager; prove the post-quantum handshake against a real certificate; sign and verify with real ML-DSA both in the production image and inside a software PKCS#11 module; run every protocol drill under real signatures; federate two instances over HTTP; type-check, test and conform the TypeScript SDK; fail the HA profile's leader, lease store and etcd member under a live write stream; and gate on CVE scans. The pattern behind them is the one Section 15 describes: every defect class found by running the system gets a permanent gate. IMPLEMENTED

Layer card: operations

What it is	The five-service stack on four substrates; HA, replicas, partitioning, backup and disaster recovery, chaos, rolling deploys; abuse controls, secrets, supply-chain gates; eighteen CI jobs and two scheduled drills.
Why it exists	A correct system that cannot survive a lost disk, a bad deploy or a hostile client is not infrastructure.
What it trusts	The drills' ephemeral hosts as a lower bound on production behaviour; the operator's placement of hosts and the custody choice.
What it refuses	A claim without a drill; a number without a stamp; a root-running container; an unpinned image; a fixable critical CVE.
What it can see	Metrics, traces and alerts about the system. The correlation id on every request never touches the audit of record.

What it cannot see	A person: quotas, alerts and metrics are keyed by agency, route and worker.
If it fails	A drill that misses its ceiling fails the build; a chaos or DR finding becomes a check; a production incident is what the readiness ledger's nine operator decisions exist to prepare for.
Checked by	`check_ha_automation`, `check_read_replica_routing`, `check_event_table_partitioning`, `check_dr_drill_scheduled`, `check_chaos_program`, `check_zero_downtime_deploy`, `check_abuse_controls`, `check_container_hardening`, `check_cve_scanning`.
Now or later	Now, on drills. Multi-region, a hardware module, and production hardware are later.
Inside and outside	Inside: everything above. Outside: the system's view of itself, which an operator needs without a view of people.

An operator running all of this needs to see what the system is doing and to understand where authority comes from, and must be able to do both without the system becoming a map of its people. The next layer is how Polaris observes and understands itself.

14 How Polaris observes and understands itself

Two cognitive layers exist, two are proposed, and one line separates all of them from the people the system serves. The design rule is stated in the repository in one sentence: make power legible, not people.

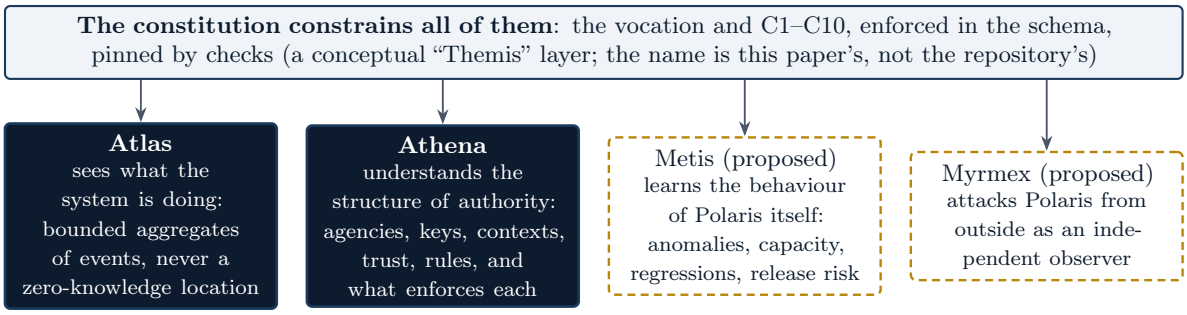


Figure 12: The cognitive layers. Atlas sees what the system is doing; Athena understands the structure of authority; Metis, proposed, would learn the behaviour of Polaris itself; Myrmex, proposed in this paper, would attack it from outside. The constitution constrains all of them. The red line is the person-prohibition: nothing above it models a natural person, and five checks fail the build if a person-level column or traversal appears in the authority layer.

14.1 Atlas sees

The Atlas is the operator's investigation surface: an analytical console that opens on a bounded, non-geographic overview, a volume time-series with failures overlaid, top-K breakdowns by context, agency, disclosure and outcome, a two-dimensional pivot, a records grid, and a map kept as a tab for what a map is good at, one region's drill or one subject's journey. Every view is a

bounded server-side aggregate, never the raw events: at most a few hundred cluster summaries for a viewport, at most 240 time buckets, at most fifty categories, keyset pagination so that a deep page stays cheap. A zero-knowledge verification is counted in volume and in the disclosure tallies and is never located, because it carries no token to attribute it by. The single-entity investigation pages are built to refuse cross-individual link analysis, and every read of an audit surface is itself recorded. A live-simulation mode is driven in a headless browser in CI and the console is asserted to actually stream. IMPLEMENTED

14.2 Athena understands structure

Athena is a read-only semantic and provenance layer over the authority tables: agencies, algorithms, contexts, trust attestations and retention policies, as ten object views and eight edge views, with four functions that answer the governance questions the tree could otherwise answer only by hand. Why may this agency issue under this algorithm? What proof and disclosure policy bounds this context? If this algorithm is deprecated, who is affected? Which mechanism enforces this constitutional rule? The constitution itself is data: C1 to C10 and the vocation are rows, and a table maps each rule to the exact live mechanism that enforces it, a trigger, a partial unique index, a named check constraint, a check function or a procedure. A check fails the build if any named mechanism no longer exists, and a database test proves each is present in the running catalogue, which closes the drift between prose and code that the constraint lattice alone could not. The signed registry of Section 12 is a view over Athena, so the map of the institutions that an outsider fetches is the same map the operator inspects. IMPLEMENTED

Athena is safe to exist only because person-legibility is made structurally impossible, not merely discouraged. Five invariants gate it, each with an adversarial detection test: no Athena view, function or table references a natural-person table or column, and no column is a stable per-person surrogate; every function is read-only, so *the graph says revoke, therefore revoked* is impossible; every function bounds its output; every authority edge resolves to an existing authority table, and the current views exclude revoked authority, so Athena describes and never owns; and every rule maps to a mechanism that exists. The same ship that created Athena removed the earlier person-aggregating views. If the prohibition ever obstructs a feature, the repository's own assessment says that is the signal to stop, not to relax it. IMPLEMENTED

14.3 What must never be built here

No person, household or relationship object. No subject handle, opaque or otherwise, stable across contexts. No traversal from a zero-knowledge event to a subject. No population segmentation or scoring. No action: the layer reads and never writes. Those are the kill criteria of the design study that endorsed a constrained Athena, and they are the boundary any future cognitive layer inherits.

14.4 Metis and Themis, named carefully

The roadmap carries a captured brief for a future, advisory-only learning subsystem that would learn Polaris itself: infrastructure anomaly detection, capacity forecasting with uncertainty, regression and release-risk intelligence, chaos learning that proposes experiments and never injects them, root-cause assistance over Athena's dependency graph, and aggregate institutional anomalies of the form *this agency normally revokes 0.8 percent a month and now revokes 4.9.* Its central hypothesis is to be falsified before any code: machine learning may infer properties of the system and must never infer, rank, score or predict the rights of natural persons, and renaming a person to a subject does not satisfy that if the data stays linkable. Its design law is advisory only: observe, model, predict, explain, recommend, and never predict then revoke, deny, issue or change a constitution or a production configuration. The brief proposes the name Metis, because the

obvious alternative collides with the metrics stack, and records that a rule that works as well as a model should be preferred to the model. Nothing of it is built; the brief calls for a full assessment first. PROPOSED

This paper uses the name Themis for the constitution's constraining role over all four layers, and the repository's own brief uses it in the same sentence: Atlas sees, Athena understands structure, Metis learns system behaviour, Themis constrains them all. Themis is not a component. It is the vocation and C1 to C10 as enforced in the schema and pinned by the checks, which already exist. The name is a reading aid and nothing in the code carries it.

Layer card: system cognition

What it is	Atlas, the bounded operational console; Athena, the read-only authority-and-constitution layer; and two proposals, Metis and Myrmex.
Why it exists	An operator must see what the system is doing and understand where authority comes from; a system this large cannot be governed by hand.
What it trusts	The append-only events and the authority tables it reads.
What it refuses	Any person-level object, handle, traversal, score or action; an unbounded read; a model where a rule would do.
What it can see	Aggregates of events and the structure of authority, keys, contexts, trust and rules.
What it cannot see	A person. The single-entity investigation page exists outside these layers, gated by role and audited on every read.
If it fails	A person reference in Athena fails five checks; an unbounded aggregate fails C8; a learning system that scored people would violate the vocation and is refused before it is designed.
Checked by	`check_athena_no_person`, `check_athena_read_only`, `check_athena_functions_bounded`, `check_athena_non_sovereign`, `check_athena_rule_enforcement_resolves`, `check_atlas_console`, the C8 caps.
Now or later	Atlas and Athena now. Metis after an assessment; Myrmex is a proposal in this paper.
Inside and outside	Inside: operations. Outside: the loop by which the system corrects its own understanding.

> *A system that can observe itself still has to learn from what it observes, and the hardest lesson Polaris has learned is that its own instruments can be green while measuring the wrong thing. The next section is that lesson.*

15 Errors become new senses: the history of self-correction

15.1 Formal truth is not intended truth

A check can be perfectly green while proving the wrong proposition. A signature can exist without ever being verified. A benchmark can report thousands of verifications per second while counting database writes. A two-witness guarantee can be certified on the strength of one witness that happened to be absent. Every one of those sentences describes a state Polaris was actually in, at a version the changelog names, and every one was found by running the system rather than reading it, or by an outside reader who did not share the author's assumptions. The project's method for this is a loop: an observation, a failure, a correction of the model of what the mechanism really proves, a new invariant with a detection test, and a permanent new sense that the failure class cannot recur unseen. Invariants accumulate and are never deleted. At `v9.345` there are 191 of them.

15.2 Selected corrections, from the record

Each row is a mechanism that was technically valid and did not prove what was intended, the version that found it, and the sense it left behind. None of these is embarrassing; each is the method working.

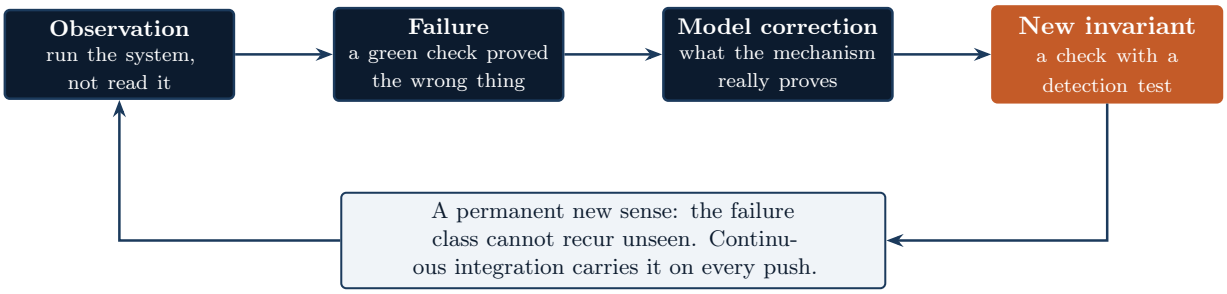


Figure 13: The loop by which an error becomes a permanent sense. Observation is running the system, not reading it; the failure is a green check that proved the wrong thing; the correction is a new statement of what the mechanism proves; the invariant is a check with a detection test; and continuous integration carries it on every push from then on.

Table 15: Corrections recorded in the changelog. The right-hand column names the check or test that now fails if the original condition returns.

Version	What was believed	What was true, and what changed	The new sense
v9.58	Tokens were post-quantum signed	The procedure wrote a placeholder string and the signing module was an unused island; signing was wired into issuance	`check_pqc_signing_wired`
v9.113	Signatures were verified	Nothing on a live path ever called verify; issuance now self-verifies and a use-path primitive exists	`check_verify_enforced`
v9.133	A verdict from one library was a verdict	One library could silently accept a forgery; a second, independent implementation must agree	`check_pqc_second_witness`
v9.257	The benchmark measured verifications per second; bulk-issued tokens were signed	It measured audit-row ingestion, about 35 times faster than cryptographic verification; bulk rows carried a placeholder literal. Three numbers are now reported separately and bulk issuance refuses an unsigned row	`signatures_cryptographically_verify`, `check_national_simulation`
v9.264	Every stored signature was two-witnessed; `usable` was current	A missing witness at issuance degraded silently to one; `usable` could be decided on a stale replica. Issuance now requires the witness and status is pinned to the primary	`check_pqc_signing_wired` (extended)
v9.272	The single-witness fast path was sound	It was sound only while the fast witness stayed trustworthy, which nothing checked; sampling through the second witness is now mandatory and a disagreement pages	`check_verify_witness_sampling`
v9.302	A transparency monitor caught a lying log	It caught a rewrite and not a split view; witnesses, gossip and the equivocation proof were added	`check_transparency_gossip`

continued on the next page

Version	What was believed	What was true, and what changed	The new sense
v9.309	The detached verifier rejected hostile input	It crashed on several classes of it; a totality contract converts every crash into the intended reject	`check_verifier_fuzz`
v9.313	The canonical oracle pinned every signed field list	Its static key-list pin used a regex that silently matched nothing; the wire specification is now checked against the signer	`check_wire_spec_matches_code`
v9.333	The exchange gateway authorized by the trust graph	It accepted an attestation from any agency on the instance, transitive trust one layer up; the responder itself must attest	`check_exchange_trust_directional`
v9.334	Long-term validation trusted a timestamp	It trusted any authentic timestamp, including the signer's own, which the signer can backdate; the verifier now names distinct trusted timestamp authorities	`check_ltv_timestamp_trust`
v9.335	A receipt alone proved an exchange	A dishonest responder can sign any receipt; the requester's envelope beside it is the evidence. A QR decoder with no bound was also made resource-bounded	`check_qr_resource_bounds`
v9.336	The broker enforced a relying party's step-up	The demand arrived only in the holder's request; it is now registered on the relying party's row, and the code is encrypted	`check_broker_policy_bound`
v9.341	Anchored time evidence survived a stolen key	A thief could mint a backdated timestamp; anchoring into a witnessed log makes a forgery one absent from every witnessed head	`check_timestamp_transparency`
v9.345	The maintainer would remember which drills a change touches	A changed verdict shipped without its drills and CI was red for four versions; a tool now names the verification a change needs from the paths and route handlers that moved	`check_ship_tool`

Two corrections in the record are about the project's own posture rather than a mechanism. At `v9.55` the repository deleted eighteen thousand lines of self-monitoring apparatus and some nine hundred invariants that asserted the apparatus's claims about itself, on the finding that the thesis was always the product and the scaffolding was never load-bearing; the constitution now says that no deeper apparatus governs the ten constraints. At `v9.332` and in the readiness pass before it, the front door was restated: the phrase *zero-knowledge identity system* was split into three claims, unlinkable verification records, a membership proof, and selective disclosure not implemented; every number was labelled measured, simulated or projected; and an extrapolation stopped being called a certification.

15.3 Multiple observers of one claim

The pattern behind the table is that Polaris increasingly refuses to let one observer decide whether a claim holds. A cryptographic verdict has two implementations. A check has a detection test that proves it can see its own violation. A drill runs the path instead of reading it. An attack tries to break the defence with the real code. A fuzzer explores the input space the drills hand-picked. An outside review, at `v9.331`, found six things the author's own instruments had not, and five

of them became invariants within eight versions. Continuous integration carries all of it on every push. None of these observers shares the others' assumptions, and that is the point: a green test is not proof that the test asked the right question, but six differently-built observers agreeing is much harder to fool than one. Section 18 proposes a seventh observer that would live outside the repository altogether.

16 What Polaris refuses to become

Powerful identity infrastructure carries risks that no amount of correctness removes, because they are risks of what a correct system can be used for. The repository names these as design and governance constraints rather than as politics, and this section states them the same way. Each is a way a working Polaris could be turned into something its constitution forbids.

Identity must not become money. A credential that carries spending authority inherits programmability, and every constraint anyone wants placed on spending accretes onto the identity token, until the system can be told whom to refuse at a fuel pump. C10 forbids a monetary claim in the schema; if a value system is ever built, it lives in a separate database that consumes verification proofs over a boundary, and the boundary is what is load-bearing. This is the single most consequential architectural decision in the repository, and it is the one that never expires with any phase.

Privacy is not merely secrecy. A system can know very little about a person and still be coercive, if every doorway requires its authorization. The zero-knowledge default bounds what is recorded; it does not bound what a society demands. The tiered-enrolment layer exists for this: exemption is a first-class, one-row state, and the not-enrolled cannot be enumerated by an accident of schema. The repository's own threat model names the strongest attack on that layer as making non-enrolment civically impossible from outside, no token, no healthcare, and says plainly that the schema cannot stop it, only make it a named act.

Technical federation is not social decentralisation. Polaris is federated in code: each authority its own key, no central store, no transitive trust. Two instances run by one author on one machine demonstrate the protocol and nothing about who runs it. Whether authorities, witnesses and relying parties are in fact independent is a property of a deployment, not of the repository, and this paper marks every such claim as requiring external validation.

Revocation must not become civil erasure. A revoked token is a terminal state in an append-only record, not a deletion; lost tokens stay lost, and their data is not erased. Succession is by reference. Mass revocation is bounded by a ceiling and a co-signer. Erasure, when a jurisdiction requires it, is pseudonymisation with an append-only record of who, when and why, never of the prior name. An authority can end a person's credential; the schema refuses to let it end the person's history.

Relying-party correlation is different from issuer unlinkability. The issuer's database cannot rebuild the zero-knowledge verification graph. Two relying parties can join their logs by the credential value. Both sentences are true at once, and the second is a permanent, documented property (Section 8). A reader who hears *unlinkable* and imagines the second has been misled, which is why the repository states it positively.

No population graph. The Atlas is bounded and never locates a zero-knowledge event; Athena is forbidden a person node, a stable subject surrogate and a subject traversal; the schema carries no attribute to filter a population by; quotas and alerts are keyed by agency, never by person. The refusal is structural because a population graph, once built, outlives the intentions of whoever built it.

Machine learning must not turn identity infrastructure into scoring. The proposed learning layer may learn how the identity system behaves and must never learn how people behave. Fraud or risk scores for individuals, behavioural profiles, relationship inference, eligibility or likelihood-of-revocation scoring, person-level clustering or embeddings, and model-driven denial of rights are presumptively prohibited, and a recommendation is evidence, never authority.

The repository also lists permanent non-goals that do not expire with any roadmap phase: no payment rails, no central biometric database, no population-scale attribute filtering or analytics, no social scoring, no commercial or advertising use of verification data, and no real personal data before a consent framework exists. Every one of them is a way the system could be made more useful to someone, and every one is refused on sight.

IN THE TOWN The laws above the town bind the town. The hall may not mint coin. The register may not be turned into a list of who is absent. The gates may not be closed on a neighbour on the word of a neighbour's neighbour. A name struck from the register stays legible as struck. And no clerk, however clever, may be set to guessing which townspeople are likely to cause trouble.

17 AI agents, proof of personhood and the future internet

This section was written as a statement of what did not exist. Most of it has since been built, and it is rewritten here as a record of what runs, what it costs, and what is still missing, with the marks distinguishing the three. The environment it describes has not changed; Polaris's answer to it has.

17.1 The problem that is arriving

When software agents can generate unlimited accounts, content, transactions, messages and apparent participants, the cheap signal that a request came from a person stops being available for free. Many digital systems will need to distinguish, without learning more than they must, between one accountable human; one authorized agent acting for that human or for an institution; anonymous machine activity; duplicated or synthetic identities; and organisational agents acting under an institution's authority. Every naive way of obtaining that signal leaks identity: accounts, phone numbers, payment instruments, biometrics. The identity question changes shape. It is no longer only *who are you*, but *what kind of actor are you, whose authority are you exercising, what are you allowed to do, and how can I verify all of that without learning anything else.* That is the future-facing question this paper considers the most important one Polaris's core could be extended to answer.

17.2 What exists today that points this way

Six ingredients of a privacy-preserving proof of personhood already run; the last three arrived after this section was first written.

- **Uniqueness at issuance.** C3, one active token per person, is a partial unique index in the database. That is the root of Sybil resistance: a human can hold one active credential, and no application code can bypass the constraint. IMPLEMENTED
- **Membership without identity.** The epoch membership proof proves that a valid token exists in an authority's committed set with public inputs of only the root, the epoch, a context and a nonce; a zero-knowledge verification records no token at all; the anonymity set is a full epoch. IMPLEMENTED
- **Revocation without naming.** A revoked token drops out of the next epoch root, so a membership proof expires with the epoch and no relying party learns who was revoked. IMPLEMENTED

- **A key the holder controls.** A credential may carry a holder public key the issuer binds to it, so possession of the file stops being possession of the credential and a verifier can require a signature from the person rather than a copy of their credential. IMPLEMENTED
- **Proving on the holder's device.** The epoch's leaf set is published, signed and bounded; every requester receives identical bytes; the holder finds their own leaf and builds the path locally. The issuer is never told which member is proving. IMPLEMENTED
- **A per-verifier nullifier.** The proof carries `Poseidon(secret || scope || epoch)`, constant for one person within one relying party's scope and epoch and uncorrelated across scopes. IMPLEMENTED

17.3 One human, once: what runs and what it costs

The membership proof stops replay of one proof, but on its own it does not stop the same person producing a thousand fresh proofs to one site and registering a thousand accounts. *One human, once, here* needs a scoped nullifier: one more public input, a hash of the holder's secret, the relying party's scope and the epoch, so a site can refuse a second proof from the same person while two sites cannot link their nullifiers. That is now a public input of the circuit. IMPLEMENTED

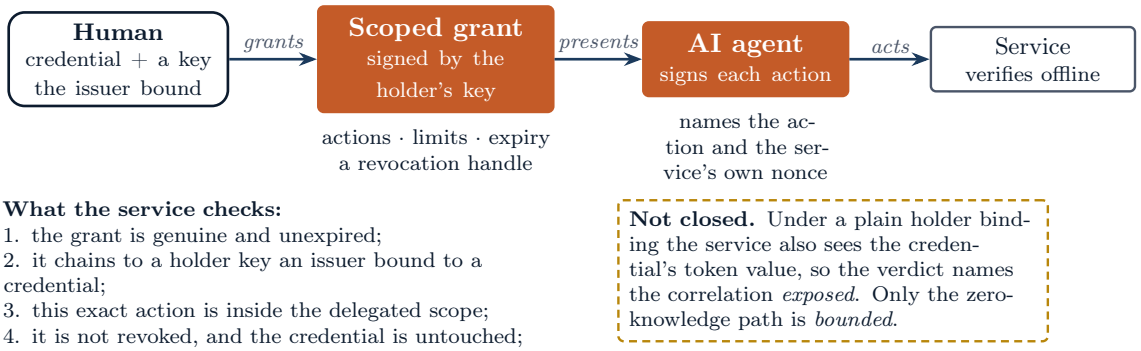
Adding it was not the small change this section once predicted, and the reason is worth recording because it is the kind of mistake a plan makes from outside the code. The epoch leaf was a `SHA3-256` digest computed *outside* the circuit and handed in as an opaque private value. A nullifier beside an opaque leaf proves only that the prover knows some number: nothing ties the two to one secret, so a prover could pair any member's leaf with a nullifier of their own choosing, and every property above would have been a claim rather than a proof. Constraining both to one secret means opening the leaf inside the circuit, and verifying a `SHA3-256` preimage there costs thousands of constraints where Poseidon costs about a hundred. So the leaf became `Poseidon(secret || context)`, a commitment the circuit opens, and the old digest became the holder's secret. The change is not backward compatible: an epoch closed under the old derivation holds leaves the current circuit cannot open, and epochs are re-closed rather than migrated.

The two conditions the vocation imposes were carried into the implementation rather than left as intentions. The nullifier's scope is per relying party and per epoch, never global, because a global nullifier is a persistent identifier and therefore the surveillance outcome the constitution refuses; and it rotates each epoch, so membership never becomes a permanent pseudonym. A relying party's ledger is keyed by its own scope and that epoch, and should not outlive it.

Two bounds remain and are stated rather than closed. The nullifier does not hide the holder from the *issuer*, which derives every leaf in order to build the tree and could therefore compute any member's nullifier in any scope; the property holds between relying parties. And the presentation layer is still full-credential: a verifier looking at a plain credential sees a stable token value, an issuer signature and a holder key, so two verifiers who deliberately keep the raw material can still correlate. What the pairwise derivation bounds is what a verifier *stores*, which is the material that gets pooled, sold, subpoenaed and breached. Section 8 states the distinction, and the verifier reports which of the two a given presentation gave rather than letting a caller assume the stronger one. The blinded and one-time presentation technologies that would remove the residue are surveyed in Section 18; none is implemented. PROPOSED

17.4 Delegating authority to an agent

The relationship is a chain, and it now runs. A human possesses a foundational credential and a key the issuer bound to it. With that key the human signs a grant giving an agent a narrowly scoped authority: named actions, stated limits, an expiry, and a revocation handle belonging to the grant alone. The agent signs each action it performs, naming the action and the service's own nonce. A service verifies, offline, that the grant is genuine and unexpired, that it covers this exact



Revoking the grant does not revoke the human. The holder signs the revocation with the same key; the issuer is never contacted and never learns the grant existed.

Figure 14: The delegation chain, as built. The human holds the foundational credential and a key the issuer bound to it. With that key they sign a grant naming actions, limits, an expiry and a revocation handle; the agent signs each action against the service's own nonce; the service verifies the chain offline. The one link drawn dashed is the residue: under a plain holder binding the service also sees the credential's token value, so the correlation it is left with is bounded rather than absent.

action, that it has not been revoked, and that it chains to a holder key an issuer bound to a real credential. IMPLEMENTED

What the design is for is visible in what it refuses. Handing an agent the credential itself gives it everything the person can do, forever, revocable only by revoking the person. The grant is the opposite of each: `actions` and `limits` sit *inside* the signed statement, so a grant widened in transit fails rather than passing invisibly; an empty action list authorises nothing and there is no way to write *everything*; a limit key the service does not recognise is refused rather than ignored, since a grant that says `max_transfers: 3` to a service that has never heard of it must not read as unlimited. And the agent must sign: without that link a grant is a bearer token, and whoever copies it in transit becomes the agent.

Revocation is the property worth naming twice. The revocation is signed by the same key that signed the grant, so anyone may publish bytes but only the holder may end it. The issuer is not contacted and never learns the grant existed, and the human's credential is untouched throughout. A person can end their agent's authority without asking permission from, or being observed by, the authority that issued their identity.

The revocation carries the grant identifier and the instant, and nothing else. It will not carry a reason. A field recording *why* a grant ended is a field a coercer can demand be filled in or left empty, and either way it turns a revocation into a signal about the person; the constitutional argument is the same one that keeps the presented code out of the holder proof's signed statement.

What a service learns is bounded, and the bound is reported rather than assumed. Under a plain holder binding the service also sees the credential's token value, so the verdict names the correlation `exposed`; only a chain carried by the zero-knowledge path, whose handle is the scoped nullifier, is named `bounded`. The grant hides the human from the agent's *actions*. It does not hide the credential from the service.

17.5 The future internet, concretely

The term Web3 is used here in one narrow sense, and it has nothing to do with currency: an internet in which people, autonomous agents, institutions, wallets, distributed services and independently operated networks need portable cryptographic trust that does not route through one

company or one state. Against that environment the following table marks what Polaris could provide and what each item's status is today.

Table 16: What a future environment would ask of Polaris, and the status of each.

The environment asks for	What Polaris has	Status
Portable credentials	A signed file any standard library verifies	IMPLEMENTED
Independently verifiable proofs	The detached verifier, two SDKs, a conformance suite	IMPLEMENTED EXTERNAL VALIDATION REQUIRED
Institutional federation	Per-authority roots, directional attestations, published feeds	IMPLEMENTED EXTERNAL VALIDATION REQUIRED
Service discovery	The signed registry	IMPLEMENTED
Document signatures with long-term validity	Signed containers with evidence fixed at the instant	IMPLEMENTED
Transparent authority behaviour	Three logs, witnesses, equivocation proofs	IMPLEMENTED EXTERNAL VALIDATION REQUIRED
Offline verification	Status assertions stapled to presentations	IMPLEMENTED
Post-quantum trust	ML-DSA, hash-based proofs, a hybrid edge; classical remainders named	IMPLEMENTED
Human, account and agent authorization relationships	A login token whose subject is per relying party, plus the agent grant below	IMPLEMENTED
Privacy-preserving uniqueness	Membership proofs carrying a per-verifier, per-epoch nullifier	IMPLEMENTED
Delegation to agents with bounded scope	A holder-signed grant: actions, limits, expiry, revocation handle	IMPLEMENTED
Revoking an agent without destroying the human's identity	A holder-signed revocation; the issuer is not contacted and the credential is untouched	IMPLEMENTED
Presentation that shows a verifier nothing stable	The zero-knowledge path only; a plain credential still shows a stable value	IMPLEMENTED PROPOSED

18 Future research and engineering

Everything in this section is marked PROPOSED unless stated otherwise. Items are ordered by how directly they bear on the future environment of Section 17.

18.1 Presentation technologies that would remove the correlator

The stable credential value in a presentation is the one property that separates Polaris from a privacy-preserving proof of personhood. Five families of technique would address it, each with a cost the repository would have to accept.

Pairwise identifiers. A different handle per relying party, derived from a holder secret and the party's identity, so that two parties cannot join their logs. Needs a holder-side secret and a rule for how the issuer's status artifacts bind to a derived handle.

Blinded or derived presentations. The holder proves possession of a signature over attributes without revealing the signature itself, so a reused signature is not a correlator. The classical constructions rely on pairing-based assumptions that a quantum computer breaks; post-quantum equivalents are an open research area, and Polaris's post-quantum posture would have to be preserved.

One-time presentation tokens. Single-use artifacts minted per presentation, at the cost of issuer contact or a pre-fetched batch, which trades against offline availability and issuer non-observation exactly as the status assertion does.

Anonymous credentials with selective disclosure. Revealing that an attribute satisfies a predicate without revealing the attribute. Polaris's disclosure levels are recorded at the issuer; a presentation-layer selective disclosure would be new machinery, and the repository explicitly says it does not claim to be one.

Zero-knowledge uniqueness. The scoped nullifier of Section 17: one added public input to the existing circuit, per relying party and per epoch, never global.

18.2 The holder-side key

Every item above, the browser bridge, agent delegation, and on-device proving share one missing piece: a key the holder controls, registered against the credential at enrolment. The current model is issuer-centric on purpose, to keep key custody, rotation and recovery from becoming a per-person problem, and the notarial signing and possession-based login follow from it. A holder key would be a credential extension with its own recovery ceremony, and the recovery layer of Section 3 is the obvious place it would attach. The sibling-path witness optimisation, already named in the roadmap, would let a holder prove membership against a national-scale tree without reconstructing it, which on-device proving requires.

18.3 Metis: learning the system, never the people

The assessment the roadmap requires before any code would have to falsify the central hypothesis (that a model can infer properties of the system without inferring the rights of people) against a synthetic-nation ground truth, compare every model to a deterministic rule, and design the constitutional invariant that would forbid the prohibited uses structurally, in the manner of Athena's five checks. The candidate uses, infrastructure anomalies, capacity forecasts with uncertainty, regression and release-risk detection, chaos learning, root-cause ranking over Athena's dependency graph, and aggregate institutional anomalies, are all about Polaris and institutions, never about citizens.

18.4 Myrmex: stigmergic debugging as an outside observer

The observers of Section 15 all live inside the repository and inherit its assumptions. This paper proposes, under the provisional name Myrmex, a debugging system that would live outside Polaris and attack it independently, inspired by how an ant colony coordinates without a central plan.

The roles would be specialised: scouts that map surfaces; trust-boundary agents that look for a claim crossing a boundary without its proof; cryptographic agents that test each verdict against the two-witness rule and the canonical discipline; privacy agents that hunt for a person-level handle or an unbounded read; state-transition agents that try to reach a state the schema should refuse; concurrency agents that race the advisory locks; protocol agents that mutate every signed artifact the way the fuzzer does but with intent; and refutation agents whose only job is to disprove what the others suspect. The queen, or orchestrator, allocates work and merges evidence and is deliberately not an omniscient authority deciding truth.

Confidence would come primarily from external evidence: a failing test, a runtime trace, a fuzz case, a static finding, a formal counterexample, an independent reproduction. Several language models agreeing is not evidence, and the design would treat it as none. The lifecycle then joins the loop the repository already runs: the colony finds suspicion; the system attempts reproduction; a reproducible failure becomes a test; the fix becomes an invariant; the former bug becomes a permanent new sense. The reason to keep it outside Polaris is the reason for the second witness: another observer, not another subsystem that inherits the assumptions it is meant to check. The repository's history contains a caution about this idea: at `v9.55` an earlier swarm apparatus that lived inside the tree and asserted claims about itself was deleted as scaffolding. Myrmex would be held to the opposite discipline, outside and adversarial, or not built.

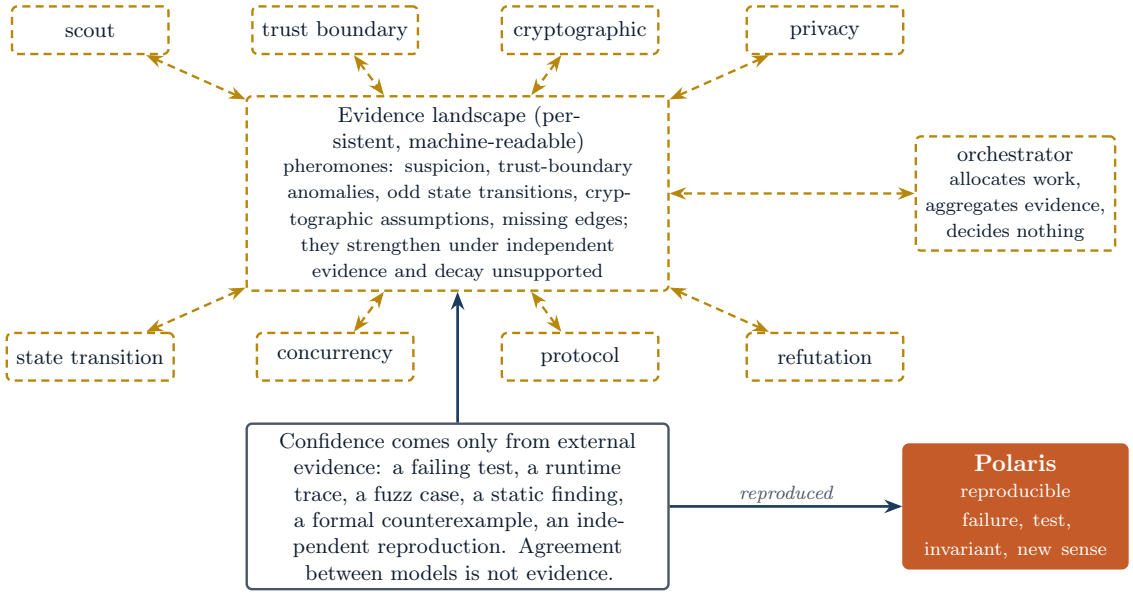


Figure 15: The proposed colony. Small specialised agents traverse the code and the architecture and leave machine-readable pheromones on a persistent evidence landscape: suspicion, a trust-boundary anomaly, an unusual state transition, a cryptographic assumption, a missing edge, an inconsistent claim, an area repeatedly associated with failure. Pheromones strengthen under independent evidence and decay unsupported; agents follow reinforced trails. An orchestrator allocates work and aggregates evidence and decides nothing. Confidence comes only from external evidence, never from agreement between models. What survives reproduction enters Polaris's own loop as a test and an invariant.

18.5 Engineering items the roadmap already names

A hash-based signer (SLH-DSA) behind the same custody interface, for a rotation away from lattice assumptions; the evaluation of a newer proof toolkit against the shipped one, gated on its stabilising and on the epoch pipeline justifying a rewrite; the epoch pipeline at scale with incremental Merkle maintenance and parallel proving; status distribution at content-delivery scale; multi-region disaster recovery; the mobile-licence and verifiable-credential format bridges, explicitly as formats and not trust models; the physical token and enrolment kit; and the external gates that are not the repository's to supply: a penetration test, a cryptographic audit, pilots with consenting users, certification, and institutionally independent witnesses.

19 Current limitations and the external validation still required

The repository's readiness ledger opens with one line, and this paper repeats it as its own bound: at `v9.345` Polaris is not production-ready for real identity data. Every engineering gap the ledger enumerated is closed and pinned by a check; what remains is not buildable in the repository. The list below separates what has not been done from what cannot be done by the author.

19.1 Decisions only a deploying organisation can make

Nine decisions must be recorded for a named deployment before real data could be held: a legal basis, impact assessment and regulator approval; the signing-key custody driver and who holds the key; the high-availability topology and its hosts; encryption at rest and its key custodian; the off-site backup target and schedule; the alerting backend, the on-call rotation and who receives the duress page; the right-to-erasure policy; the retention schedule per class, with the counsel who says the number satisfies the statute; and an independent penetration test with a

threat-model sign-off. Polaris records each decision and its justification; it does not know the law. EXTERNAL VALIDATION REQUIRED

19.2 What is not built

- The physical token is modelled, not manufactured; the enrolment kit, the card profile and the verifier device are roadmap rows.
- No hardware security module has been used; the PKCS#11 driver is proven against a software token, and the lattice library is not FIPS-validated.
- The profile is single-region; multi-region recovery, status distribution at content-delivery scale, and the epoch pipeline at national scale are not built.
- A hash-based signer is registered and not wired. Two internal transport hops, the certificate signatures and today's operator authenticators remain classical.
- Blinded and one-time presentations are not built, so a verifier looking at a full credential still sees a stable token value, an issuer signature and a holder key. The per-relying-party handle bounds what a verifier *stores*, not what it is *shown*; Section 8 states the distinction and the verifier reports which of the two a given presentation gave. This is the residue of the holder-side work, not the whole of it: the holder key, on-device proving, the scoped nullifier and the pairwise handle were built at `v9.349` through `v9.353` and this list said otherwise until `v9.355`.
- Neither the scoped nullifier nor the pairwise handle hides a holder from the *issuer*, which derives every epoch leaf in order to build the tree and could therefore compute any member's nullifier in any scope. Issuer-blind derivation is a different construction and is not claimed.
- The interim credential during a recovery cool-down is not built.

19.3 What has not been validated by anyone else

- No external penetration test, cryptographic audit, red team, bug bounty or certification has occurred. The scope document for the engagement exists; no engagement has been commissioned.
- No pilot with consenting users has run; the data has always been notional.
- No external team has implemented the protocol from the specification and conformance suite alone; the suite is the contract and the integration has not happened.
- No institutionally independent witness watches any log; the witnesses in the record are processes in a drill. Two federated instances are two instances of one code base run by one author.
- The circuit over the proof library has had no external review, and the concrete bit security of the shipped proof configuration has not been independently derived.
- The duress path is a tested property of the modelled flow, not an audited side-channel guarantee; long-run frequency analysis is an accepted residual.
- The repository's own thesis, that a stranger can read it cold and correctly infer what it needs, was never tested by a stranger before its deadline and is recorded as inconclusive, with the strong claim retired.

19.4 What the numbers are

Every performance number in this paper is single-node on one laptop, or a simulation at a stated scale factor, or a projection that the repository labels as one. No production cluster has been measured. The national planning targets are targets.

19.5 Documentation that lags the code

Researching this version found a few places where the repository's own prose is behind its code or its history; they are recorded in the author's notes that accompany this version rather than fixed silently. Where this paper and any repository document disagree, the paper followed the code at `v9.345` and says so in the note.

20 Conclusion: the whole organism

A reader who has followed the rings outward should now be able to answer the questions this paper set out to make answerable. Each answer below is one sentence, and each points back to the section that carries the mechanism.

What the central credential is. One signed claim, *this authority issued this token to this person*, held as a file the holder controls, in a lifecycle with exactly one active state (Section 3).

Why the schema surrounds it. Because a promise enforced only by the people who run the system is not a promise; triggers, check constraints and unique indexes bind every client and every restore (Section 4).

How cryptographic trust works. Post-quantum signatures over digests, checked by two independent implementations at issuance and sampled at use, with keys that have a recorded lifecycle (Section 5).

How verification differs from authorization. Authenticity is permanent and cacheable; authorization is fresh and revocable, decided on the primary or by a short-lived signed assertion whose window is the honest cost of going offline (Sections 6 and 7).

How privacy is bounded. By what is stored, not by who may read it: no token identifier on a zero-knowledge event, a membership proof that reveals nothing but membership, bounded reads, and a stated limitation on cross-verifier correlation (Section 8).

How relying parties use credentials. Through a wallet protocol, a detached verifier, two SDKs and a login broker that keeps no record of who logged in where (Section 9).

How authorities federate. Each with its own key and no central store, by directional per-context attestations that never form a chain, and by signed feeds a relying party decides on offline (Section 10).

How transparency constrains authority. Append-only logs with signed heads, monitors that prove history only grew, and witnesses that cosign, gossip, and produce a non-repudiable proof when a log tells two stories (Section 11).

How institutional exchange works. Through a signed registry, a gateway that authenticates by key and authorizes by the responder's own attestation, receipts that commit to hashes and never bodies, a timestamp authority that learns nothing, and signatures that outlive their keys (Section 12).

How Polaris survives failures. By drilling them: failover, replicas, partitions, backup and restore, chaos and rolling deploys, each with a measured number and a ceiling, on every push (Section 13).

What Atlas and Athena do. Atlas sees what the system is doing in bounded aggregates; Athena understands the structure of authority and the constitution as data; neither may model a person (Section 14).

Why AI agents change the identity problem. Because the question becomes what kind of actor is acting, under whose authority, within what scope, verifiable without learning anything else (Section 17).

How privacy-preserving proof of personhood might fit. On the existing uniqueness constraint and membership proof, with a scoped nullifier, a holder-side key and a presentation that carries no stable handle, none of which is built (Sections 17 and 18).

How delegated agent authority might work. As a signed, scoped, expiring, separately revocable grant from a human's credential to an agent, verified by a service that learns only the five answers it needs (Section 17).

How stigmergic debugging could help. As an outside colony of specialised agents leaving evidence on a shared landscape, with confidence drawn only from reproductions, feeding the loop by which a reproducible failure becomes a permanent invariant (Section 18).

What Polaris can do today. Issue, sign, verify, revoke, present, federate, log, witness, exchange, timestamp and sign documents, all on notional data, all drilled in continuous integration, between instances of one code base (Appendix A).

What it cannot do today. Hold real identity data, run on hardware tokens or a hardware module, prove independence of institutions or implementations, prevent cross-verifier correlation, or claim any property no outside party has tested (Section 19).

What it is ultimately trying to become. Identity infrastructure in which no important claim has to be taken on the word of a central application, because authority, state, history, privacy and the system's own behaviour are exposed to independent verification and constrained by layers that do not share each other's assumptions.

The town's register is small. The walls around it are what make it safe to keep. And the reason the system has so many rings is not that its builders enjoyed rings: each one was added when the ring inside it was found to be correct and still unable to prove what a society would need to know. Credential authenticity is not current authorization. Current authorization is not privacy. Privacy is not freedom from coercion. Federation is not institutional independence. A transparency log is not protection from a split view. And a green test is not proof that the test asked the right question. Polaris is the accumulation of those sentences into mechanisms, and the honest map of where the accumulation stands: at version 9.345 for the body of this paper, and at version 9.355 for the holder-side sections, which were rewritten when the work they described as missing was built.

Appendix A Capability status

The table answers, for every major subsystem, whether it runs, what limits it, and which invariant check fails the build if it stops being true. The marks are those of the front matter. Most rows were measured at `v9.345`; the holder-side rows were rewritten at `v9.355`, when Phase P9 moved five of them from proposal to running code. No row moved the other way.

Table 17: Capability status. A row carrying two marks runs today and still requires external validation for the stronger claim.

Capability	Status	Limit stated by the repository	Pinned by
Schema constraints, C1–C10	IMPLEMENTED	The database owner is outside the threat model	`check_c1c10_objects_resolve`
Append-only audit of record (13 instances)	IMPLEMENTED	All thirteen enforced by trigger since `v9.347`	`check_aor_append_only_triggers`
Retention as data with a 365-day floor	IMPLEMENTED	Archive and purge remain deliberate operator steps	`check_retention_engine`

continued on the next page

Capability	Status	Limit stated by the repository	Pinned by
ML-DSA-65 and ML-DSA-87 signing	IMPLEMENTED	Placeholder profile in development; SLH-DSA registered, not wired	`check_pqc_signing_wired`
Two-witness verification, sampled at use	IMPLEMENTED	Catches divergence, not a shared misreading	`check_pqc_second_witness`, `check_verify_witness_sampling`
Key custody: file, PKCS#11, KMS	IMPLEMENTED EXTERNAL VALIDATION REQUIRED	Software token in CI, no hardware module	`check_key_custody_abstraction`
Key register and signed trust list	IMPLEMENTED	Rotation of a live key file or slot is an operator ceremony	`check_trust_lifecycle`
Detached verifier, fuzzed and attacked	IMPLEMENTED	Needs the prover binary for the zero-knowledge path only	`check_detached_verifier`, `check_verifier_fuzz`, `check_attacks_run`
Python and TypeScript SDKs, conformance suite	IMPLEMENTED EXTERNAL VALIDATION REQUIRED	No external implementation from the specification	`check_conformance_suite`, `check_typescript_sdk`
Frozen version 1, cross-version suite	IMPLEMENTED		`check_frozen_v1`
Relying-party verify API	IMPLEMENTED	Client-credentials only; mutual TLS deferred	`check_relying_party_api`
Signed status assertion, offline authorization	IMPLEMENTED	The revoked-but-valid window is the cost	`check_offline_verification`
Epoch checkpoints, revocation feeds, status bundle	IMPLEMENTED	Distribution at content-delivery scale not built	`check_epoch_revocation_propagation`, `check_federation_status_bundle`
Zero-knowledge membership proof, second witness	IMPLEMENTED EXTERNAL VALIDATION REQUIRED	No external audit; holder proves locally; scoped nullifier built	`check_zk_tree_depth_synced`, `check_scoped_nullifier`, the witness differential
Issuer-side unlinkable verification records	IMPLEMENTED	Scoped to zero-knowledge mode	the C2 CHECK, the redaction suite
Cross-verifier correlation, bounded	IMPLEMENTED	Stored handles are per-verifier; a plain presentation still shows stable material	`check_pairwise_presentation`
Wallet protocol, presentations, QR framing	IMPLEMENTED	No native clients, no browser bridge	`check_wallet_presentation`
Auth broker with registered policy	IMPLEMENTED	No session product, no consent screen	`check_auth_broker`, `check_broker_policy_bound`
Federation: roots, attestations, manifests	IMPLEMENTED EXTERNAL VALIDATION REQUIRED	Attestations operator-recorded; two instances by one author	`check_federation_topology`, `check_federation_two_instances`
Cross-authority zero-knowledge	IMPLEMENTED	Abstains without the binary; issuer bound by root	`check_cross_authority_zk`
Transparency logs, monitor	IMPLEMENTED	A monitor alone cannot see a split view	`check_transparency_log`
Witnesses, gossip, equivocation proof	IMPLEMENTED EXTERNAL VALIDATION REQUIRED	Witnesses are drill processes; independence is a deployment	`check_transparency_gossip`
External-ledger publication	IMPLEMENTED PROPOSED	File backend only; chain drivers declared	`check_transparency_publication`
Signed registry	IMPLEMENTED		`check_registry`
Exchange gateway and receipts	IMPLEMENTED	JSON bodies; no streaming; the responder's own instance only	`check_exchange_gateway`, `check_exchange_receipt`

continued on the next page

Capability	Status	Limit stated by the repository	Pinned by
Timestamp authority, optional anchoring	IMPLEMENTED	Anchor verification not in the SDKs	`check_timestamp_authority`, `check_timestamp_transparency`
Document signing, long-term validation	IMPLEMENTED	Notarial model; signs on the holder's behalf, not with their key	`check_document_signing`, `check_ltv_timestamp_trust`
Protocol versioning, algorithm agility	IMPLEMENTED	PKCS#11 and KMS drivers ML-DSA-65 only	`check_protocol_versioning`, `check_algorithm_migration`
HA, replicas, partitioning, DR, chaos, rolling deploys	IMPLEMENTED	Ephemeral CI hosts; single region	`check_ha_automation` and its siblings
Post-quantum TLS edge	IMPLEMENTED	Opportunistic; internal hops classical	`check_edge_pq_kex`
Abuse controls, session and origin hardening, WebAuthn MFA	IMPLEMENTED	Authenticators classical until hardware ships	`check_abuse_controls`, `check_session_origin_hardening`
Atlas	IMPLEMENTED		`check_atlas_console`, the C8 caps
Athena	IMPLEMENTED	Read-only; no person node by five checks	`check_athena_no_person`
Metis (learning the system)	PROPOSED	Assessment required before code	
Myrmex (stigmergic debugging)	PROPOSED	Proposed in this paper	
Holder key bound to a credential	IMPLEMENTED	Optional; the proof deliberately does not cover the presented code	`check_holder_key_binding`
Holder-side proving, published anonymity set	IMPLEMENTED	A swapped set is refused, not annotated	`check_holder_side_prover`, `check_commitment_mismatch_is_a_refusal`
One human, once, per relying party	IMPLEMENTED	Does not hide the holder from the ISSUER; resets each epoch	`check_scoped_nullifier`
Agent delegation, revocable by the holder	IMPLEMENTED	A plain binding still shows the service the credential	`check_agent_grant`
Blinded or one-time presentations	PROPOSED	The residue: a plain credential shows a verifier stable material	
Hardware token, enrolment kit	PROPOSED	Roadmap phase	
Pilots, certification, external audit	EXTERNAL VALIDATION REQUIRED	External actors	

Appendix B The metacognitive map

The ladder below is the paper's answer to the request for an explicit map of Polaris as layers of knowledge and scepticism. Each rung states what the layer knows and what it refuses to take on faith from the rung below. The layers are not exactly the rings of Figure 1, because some rings hold two kinds of knowledge; the rungs follow the implementation.

Table 18: The layers, what checks each, and whether each is needed now or is preparation.

Layer	Name	Checked by	Now or later
L0	Human subject	Nothing in the software; the enrolment tiers keep non-enrolment a named state	Now
L1	Credential core	C3, the signature triggers, the lifecycle trigger	Now
L2	Structural constraints	The audit-of-record and cascade checks, the CHECK-constraint suite	Now

continued on the next page

Layer	Name	Checked by	Now or later
L3	Cryptographic evidence	The two-witness checks, the canonical oracle, the attack suite	Now
L4	Verification and status	The detached-verifier, conformance, offline-verification and status checks	Now
L5	Privacy and disclosure	The C2 CHECK, the redaction suite, the Athena person checks	Now
L6	Federation	The topology, manifest, feed and two-instance checks	Now as protocol; later as institutions
L7	Transparency	The log, gossip and publication checks	Now as protocol; later as independent witnesses
L8	Institutional services	The registry, gateway, receipt, timestamp and signing checks	Now between instances
L9	Operations	The HA, replica, partition, DR, chaos and hardening checks	Now on drills; later on production hardware
L10	System cognition	The Atlas caps, the five Athena checks	Atlas and Athena now; Metis, Myrmex later
L11	Societal boundary	C10's structural absence, the non-goals, the vocation	Now, and permanent
L12	Agent and web environment	Nothing yet	Later

Appendix C Counts at v9.345, and how they were measured

Every count in this paper was taken from the tree at tag `v9.345` (commit `c20a3d4`, 9 September 2026) by the command beside it, not copied from an earlier document. Where the repository's own stamped number differs in kind (a test count measured at an earlier version), the difference is stated.

Table 19: Counts at HEAD and their method.

Quantity	Count	Method
Canonical tables in `01_schema.sql`	36	The file declares 40 `CREATE TABLE` statements, of which four are the default partition children of the four partitioned event tables. A migrated deployment holds 43 tables.
Triggers in `06_triggers.sql`	26	`grep -c 'CREATE TRIGGER'`
Audit-of-record instances	13	The table in `docs/design/audit-of-record.md`, plus the partially enforced `RecoveryRequest`
Invariant checks	191	`grep -c '^def check_' polaris_checks/checks.py`; each has a detection test in `test_checks.py`
Application routes	119	`grep -c '^@app.route(' polaris_web/app.py`
CI jobs in `ci.yml`	18	The job keys of the workflow; two more workflows run monthly and weekly
Signed wire formats in the specification	20	Distinct `polaris-*/1` strings in `docs/reference/WIRE-SPEC.md`
Conformance cases	48	`len(cases)` in `conformance/cases.json`; the frozen version-1 set holds 44
Drill scripts	35	Files matching `scripts/polaris-*drill.*`
Tagged versions	292	`git tag`, from `v9.30` (16 May 2026) to `v9.345` (9 September 2026)

continued on the next page

Quantity	Count	Method
Test functions defined	1,048	`grep -c 'def test_'` across the product, CLI, check-layer and SDK suites; a definition count, not a run count
Product tests passing, as stamped by the README	755	Measured at `v9.332` on the reference machine; 15 skip without optional backends
Crypto witness tests, as stamped	84 of 89	Measured at `v9.332`; 5 need a PKCS#11 token or a KMS key and run in CI

The paper's performance numbers are those of Table 12, each with the version that measured it. The reference machine throughout the repository is an Apple Silicon laptop with eight cores and sixteen gigabytes of memory.

Appendix D Glossary

Anchor set. The published issuer keys a relying party has chosen to trust. Trust in Polaris is always membership in the verifier's own anchor set, never a decision the issuer makes for it.

Append-only. A table on which UPDATE and DELETE are refused by a database trigger, so history can only be added to. The audit of record is the named pattern of applying this per table.

Attestation. A directional, per-context statement that one agency accepts another's credentials, recorded as a row. Never transitive.

Authenticity pack. A credential as a file: the token value, its signature and the issuer's public key. What a holder holds and a stranger verifies.

Canonicalisation. Turning a structure into one exact sequence of bytes (sorted keys, no whitespace) so that a signature made on one machine can be checked on another that rebuilds the same bytes.

Consistency proof. A short proof that a smaller Merkle tree is a prefix of a larger one, so that a log only appended between two heads.

Detached verifier. A program that checks Polaris artifacts with no Polaris code, no database and no network: `scripts/polaris-verify.py`.

Disclosure level. What a verification records: zero-knowledge (no token identifier), selective (named attributes), or full (the identity, logged).

Epoch. A closed, immutable commitment to the active-token set at a moment, as a Merkle root, against which membership proofs are made.

Equivocation. A log telling two observers two different histories. The proof of it is two signed heads of the same size with different roots.

Federation. Independent authorities, each its own root, recognising one another through explicit attestations, with no central service.

Gossip. Witnesses sharing the log heads they were shown so that a split view is detected by comparison.

High availability. A database that survives the loss of its leader by promoting a replica under a lease, drilled and measured.

Inclusion proof. A short proof that one entry is in a Merkle tree with a given root.

Long-term validation. Deciding that a signature was valid at the instant it was made, from evidence fixed at that instant, so that the decision survives the key's later retirement.

	WHAT THE LAYER KNOWS	WHAT IT REFUSES TO TAKE ON FAITH
L0 Human subject	exists outside the software	the software's description of the person
L1 Credential core	one active token, one signature, one person	any state the schema cannot represent
L2 Structural constraints	which states may exist at all	application code promising to behave
L3 Cryptographic evidence	who signed what, checkable by anyone	a single implementation's verdict
L4 Verification and status	authentic, and authoritative right now	authenticity as if it were authorization
L5 Privacy and disclosure	what a verifier is allowed to learn	the client's claim about its own disclosure level
L6 Federation	whom an authority explicitly trusts, in which context	transitive trust, a central authority
L7 Transparency	that history was not rewritten or forked	a single observer's view of the log
L8 Institutional services	that an exchange happened and was authorized	keeping the message to prove it
L9 Operations	that the system survives failure and load	a claim that was not drilled
L10 System cognition	what the system is doing and how power is structured	anything about a person
L11 Societal boundary	what Polaris refuses to become	money, scoring, a required doorway
L12 Agent and web environment	what kind of actor is acting, under whose authority	a stable handle shown to everyone (open problem)

ZOOM: ATOM, ORGAN, ORGANISM, TOWN, FEDERATION, ECOSYSTEM

Figure 16: The ladder. From the person, who exists outside the software, to the environment of agents the system is preparing for. The right-hand column is the refusal each layer makes of the layer below it; the last rung's refusal is an open problem rather than a mechanism.

Merkle tree. A cryptographic fingerprint of a whole set, built by hashing pairs upward to one root, that lets a small proof show one item belongs to the set.

ML-DSA. The lattice-based post-quantum signature scheme standardised as FIPS 204; parameter sets 65 and 87 are accepted.

Nullifier. In a zero-knowledge system, a value derived from a secret and a scope that lets a verifier detect a second use without linking uses across scopes. Not implemented in Polaris.

Pairwise identifier. A different handle for each relying party, so that two parties cannot join their logs. Not implemented in Polaris.

Partition. A table split into monthly pieces by timestamp, transparent to queries, so that old months can be detached to the archive.

PKCE. A proof-of-possession extension to the authorization-code login flow, so that an intercepted code cannot be exchanged by an attacker.

Post-quantum. Cryptography whose security does not rest on problems a quantum computer solves efficiently.

Replica. A copy of the database that follows the primary with some lag; safe for reading immutable material, unsafe for deciding current authorization.

Revocation feed. A signed list of the hashes of revoked credentials an authority issued, consumed offline.

Signed tree head. A log's signed commitment to its whole history at a size.

Status assertion. A short-lived, issuer-signed statement of a credential's current status, stapled to a presentation so that authorization is decidable offline.

Trust list. A signed statement of every key an instance knows, with each key's status and the instants those statuses took effect.

Two-witness principle. No cryptographic verdict is trusted from a single implementation; a second, independently written one must agree or abstain explicitly.

Witness. An independent party that cosigns log heads it found consistent and refuses a fork.

Zero-knowledge proof. A proof that a statement is true which reveals nothing beyond the statement's truth; in Polaris, membership of a token in an epoch.

Appendix E From Version 1 to Version 2

Version 1 remains the historical report and is not edited. This appendix records what Version 2 kept, what it corrected, and what it retired, so that a reader of both can reconcile them.

Kept

- The token-centred entity geometry (Figure 2), reproduced from Version 1 without change and used as the nucleus of every map in this paper.
- The lifecycle state machine (Figure 3), reproduced without change.
- The disclosure-level argument: that the level is a persisted property of the verification record, enforced by a check constraint, and that this is what prevents the verification log from functioning as a surveillance database.
- The argument that post-quantum signing belongs on the first day, through the secrecy horizon of identity data and Mosca's inequality, and the substrate argument that every higher property of an identity system is derivative of its cryptographic primitives. Both are summarised here and stand in Version 1 in full.
- The comparison of design properties against deployed systems, which Version 2 does not repeat; the repository's front page carries the current table and marks Polaris as neither deployed nor issuing at national scale.
- The threat-model frame (STRIDE), which the repository's design record now carries with every threat mapped to a control or an explicit acceptance.

Corrected

- Twelve tables became thirty-six. The three subsidiary entities of Version 1 (device bindings, ledger anchors, the revocation list) remain; the rest of the growth is described ring by ring in Section 4.
- Version 1 stated that the audit tables were append-only by convention and tooling and that the state machine would be enforced by a trigger in a production deployment. Both are now enforced by triggers, and a check fails the build if either stops being.

- Version 1 described the ledger anchor as an optional decentralised-identifier path. The off-chain batch commitment exists and is monitored as a transparency log; publication to an external chain remains an operator action with declared drivers and no live integration.
- Version 1 modelled seven verification contexts with per-context biometric requirements and a physical token with on-device biometric binding. The contexts remain; the physical token remains modelled and not manufactured, and this paper marks it as a proposal.
- The two speculative binding horizons of Version 1's final appendix (genomic and quantum-observer binding) are not carried forward as design. The repository's readiness pass quarantined the speculative schema behind a check that it cannot return, and this paper treats the holder-side key, not a new biometric, as the extension that the future environment needs.

Added

Everything from Section 5 onward describes machinery that did not exist when Version 1 was written: real signing and custody, the two witnesses, the detached verifier and the conformance contract, status and offline authorization, the membership proof and its second witness, the wallet protocol and the broker, federation, transparency, the exchange fabric, the operational drills, Atlas and Athena, the history of self-correction, the societal boundaries, and the future-facing sections on agents, personhood, delegation and outside observers.